

Introducción a los Agentes y Arquitecturas de Agentes

****Autores:**** Alan Blount, Antonio Gulli, Shubham Saboo, Michael Zimmermann y Vladimir Vuskovic

****Introducción a los Agentes y Arquitecturas de Agentes****

****Noviembre de 2025****

Agradecimientos

****Colaboradores de contenido****

- Enrique Chan
- Mike Clark
- Derek Egan
- Anant Nawalgaria
- Kanchana Patlolla
- Julia Wiesinger

****Curadores y editores****

- Anant Nawalgaria
- Kanchana Patlolla

****Diseñador****

- Michael Lanning

Tabla de contenidos

De la IA Predictiva a los Agentes Autónomos	6
Introducción a los Agentes de IA	8
El Proceso de Resolución de Problemas de Agentes	10
Una Taxonomía de Sistemas de Agentes	14
Nivel 0: El Sistema de Razonamiento Central	15
Nivel 1: El Solucionador de Problemas Conectado	15
Nivel 2: El Solucionador de Problemas Estratégico	16
Nivel 3: El Sistema Multiagente Colaborativo	17
Nivel 4: El Sistema Autoevolutivo	18
Arquitectura Central de Agentes: Modelo, Herramientas y Orquestación	19
Modelo: El "Cerebro" de tu Agente de IA	19
Herramientas: Las "Manos" de tu Agente de IA	20
Recuperación de Información: Anclaje en la Realidad	21
Ejecución de Acciones: Cambiando el Mundo	21
Llamada a Funciones: Conectando Herramientas a tu Agente	22

La Capa de Orquestación	22
Decisiones de Diseño Central	23
Instruir con Conocimiento del Dominio y Persona	23
Aumentar con Contexto	24
Sistemas Multiagente y Patrones de Diseño	24
Despliegue de Agentes y Servicios	26
Agent Ops: Un Enfoque Estructurado para lo Impredecible	27
Medir lo que Importa: Instrumentación del Éxito como un Experimento A/B	29
Calidad en lugar de Aprobado/Reprobado: Usando un Juez LM	29
Desarrollo Impulsado por Métricas: Tu Go/No-Go para el Despliegue	30
Depurar con Trazas de OpenTelemetry: Respondiendo "¿Por qué?"	30
Valorar los Comentarios Humanos: Guiando tu Automatización	31
Interoperabilidad de Agentes	31
Agentes y Humanos	32
Agentes y Agentes	33
Agentes y Dinero	34
Tabla de contenidos	
Asegurar un Agente Individual: El Compromiso de Confianza	34
Identidad del Agente: Una Nueva Clase de Principal	35
Políticas para Restringir el Acceso	37
Asegurar un Agente ADK	37
Escalar de un Agente Individual a una Flota Empresarial	39
Seguridad y Privacidad: Fortaleciendo la Frontera de Agentes	40
Gobernanza de Agentes: Un Plano de Control en lugar de Expansión	40
Cómo evolucionan y aprenden los agentes	42
Cómo aprenden y evolucionan los agentes	43
Simulación y Gimnasio de Agentes - la próxima frontera	46
Ejemplos de agentes avanzados	47
Google Co-Científico	47
Agente AlphaEvolve	49
Conclusión	51
Notas Finales	52

De la IA Predictiva a los Agentes Autónomos

La inteligencia artificial está cambiando. Durante años, el enfoque ha estado en modelos que destacan en tareas pasivas y discretas: responder una pregunta, traducir texto o generar una imagen a partir de una instrucción. Este paradigma, aunque poderoso, requiere constante dirección humana para cada paso.

Ahora estamos viendo un cambio de paradigma, pasando de la IA que simplemente predice o crea contenido a una nueva clase de software capaz de resolución autónoma de problemas y ejecución de tareas.

Esta nueva frontera se construye en torno a los agentes de IA. Un agente no es simplemente un modelo de IA en un flujo de trabajo estático; es una aplicación completa que hace planes y toma acciones para lograr objetivos. Combina la capacidad de razonamiento de un Modelo de Lenguaje (LM) con la capacidad práctica de actuar, permitiendo

Los agentes son la evolución natural de los Modelos de Lenguaje, hechos útiles en el software.

que manejen tareas complejas y de múltiples pasos que un modelo solo no puede. La capacidad crítica es que los agentes pueden trabajar por su cuenta, descubriendo los siguientes pasos necesarios para alcanzar un objetivo sin una persona guiándolos en cada momento.

Este documento es el primero de una serie de cinco partes, actuando como una guía formal para los desarrolladores, arquitectos y líderes de producto que hacen la transición de pruebas de concepto a sistemas de agentes robustos y de nivel de producción. Si bien construir un prototipo simple es sencillo, garantizar la seguridad, calidad y confiabilidad es un desafío significativo. Este documento proporciona una base integral:

- Anatomía Central: Descomponiendo un agente en sus tres componentes esenciales: el modelo de razonamiento, las herramientas accionables y la capa de orquestación gobernante.
- Una Taxonomía de Capacidades: Clasificando agentes desde solucionadores de problemas conectados simples hasta sistemas multiagente colaborativos complejos.
- Diseño Arquitectónico: Profundizando en las consideraciones de diseño prácticas para cada componente, desde la selección del modelo hasta la implementación de herramientas.
- Construcción para Producción: Estableciendo la disciplina Agent Ops necesaria para evaluar, depurar, asegurar y escalar sistemas de agentes desde una única instancia hasta una flota con gobernanza empresarial.

Construyendo sobre el documento blanco anterior de Agents¹ y el Agent Companion²; esta guía proporciona los conceptos fundamentales y los marcos estratégicos que necesitará para construir, desplegar y gestionar con éxito esta nueva generación de aplicaciones inteligentes que pueden razonar, actuar y observar para lograr objetivos³.

Introducción a los Agentes de IA

Las palabras son insuficientes para describir cómo los humanos interactúan con la IA. Tendemos a antropomorfizar y usar términos humanos como "pensar" y "razonar" y "saber". Aún no tenemos palabras para "saber con significado semántico" frente a "saber con alta probabilidad de maximizar una función de recompensa". Esos son dos tipos diferentes de saber, pero los resultados son los mismos el 99.X% de las veces.

En los términos más simples, un Agente de IA puede definirse como la combinación de modelos, herramientas, una capa de orquestación y servicios de tiempo de ejecución que utiliza el LM en un bucle para lograr un objetivo.

Estos cuatro elementos forman la arquitectura esencial de cualquier sistema autónomo.

- El Modelo (El "Cerebro"): El modelo de lenguaje (LM) o modelo fundamental central que sirve como motor de razonamiento del agente para procesar información, evaluar opciones y tomar decisiones. El tipo de modelo (de propósito general, ajustado o multimodal) dicta las capacidades cognitivas del agente. Un sistema de agentes es el curador definitivo de la ventana de contexto de entrada del LM.
- Herramientas (Las "Manos"): Estos mecanismos conectan el razonamiento del agente con el mundo exterior, permitiendo acciones más allá de la generación de texto. Incluyen extensiones de API, funciones de código y almacenes de datos (como bases de datos o almacenes vectoriales) para acceder a información factual en tiempo real. Un sistema de agentes permite que un LM planifique qué herramientas usar, ejecute la herramienta y ponga los resultados de la herramienta en la ventana de contexto de entrada de la siguiente llamada LM.
- La Capa de Orquestación (El "Sistema Nervioso"): El proceso de gobierno que gestiona el bucle operativo del agente. Maneja la planificación, la memoria (estado) y la ejecución de la estrategia de razonamiento. Esta capa utiliza marcos de instrucción y técnicas de razonamiento (como Cadena de Pensamiento⁴ o ReAct⁵) para descomponer objetivos complejos en pasos y decidir cuándo pensar versus usar una herramienta. Esta capa también es responsable de dar a los agentes la memoria para "recordar".
- Despliegue (El "Cuerpo y Piernas"): Si bien construir un agente en una laptop es efectivo para prototipar, el despliegue en producción es lo que lo convierte en un servicio confiable y accesible. Esto implica alojar el agente en un servidor seguro y escalable e integrarlo con servicios de producción esenciales para monitoreo, registro y gestión. Una vez desplegado, el agente puede ser accedido por usuarios a través de una interfaz gráfica o programáticamente por otros agentes a través de una API Agente-a-Agente (A2A).

Al final del día, construir un agente de IA generativa es una nueva forma de desarrollar soluciones para resolver tareas. El desarrollador tradicional actúa como un "albañil", definiendo precisamente cada paso lógico. El desarrollador de agentes, en cambio, es más como un director. En lugar de escribir código explícito para cada acción, configuras el escenario (las instrucciones guía y las instrucciones), seleccionas el elenco (las herramientas y APIs) y proporcionas el contexto necesario (los datos). La tarea principal se convierte en guiar a este "actor" autónomo para que entregue el rendimiento deseado.

Rápidamente descubrirás que la mayor fortaleza de un LM: su increíble flexibilidad, también es tu mayor dolor de cabeza. La capacidad de un modelo de lenguaje grande para hacer cualquier cosa hace que sea difícil obligarlo a hacer una cosa específica de manera confiable y perfecta. Lo que solíamos llamar "ingeniería de instrucciones" y ahora llamamos "ingeniería de contexto" guía a los LMs para generar el resultado deseado. Para cualquier llamada única a un LM, ingresamos nuestras instrucciones, hechos, herramientas disponibles para llamar, ejemplos, historial de sesión, perfil de usuario, etc., llenando la ventana de contexto con la información adecuada para obtener las salidas que necesitamos. Los agentes son software que gestionan las entradas de los LMs para hacer el trabajo.

La depuración se vuelve esencial cuando surgen problemas. "Agent Ops" esencialmente redefine el ciclo familiar de medición, análisis y optimización del sistema. A través de trazas y registros, puedes monitorear el "proceso de pensamiento" del agente para identificar desviaciones de la ruta de ejecución prevista. A medida que los modelos evolucionan y los marcos mejoran, el rol del desarrollador es proporcionar componentes críticos: experiencia en el dominio, una personalidad definida e integración perfecta con las herramientas necesarias para la finalización práctica de tareas. Es crucial recordar que las evaluaciones y valoraciones integrales a menudo superan la influencia de la instrucción inicial.

Cuando un agente está configurado con precisión con instrucciones claras, herramientas confiables y un contexto integrado que sirve como memoria, una excelente interfaz de usuario, la capacidad de planificar y resolver problemas, y conocimiento general del mundo, trasciende la noción de mera "automatización de flujos de trabajo". Comienza a funcionar como una entidad colaborativa: un miembro de tu equipo altamente eficiente, singularmente adaptable y notablemente capaz.

En esencia, un agente es un sistema dedicado al arte de la curación de ventanas de contexto. Es un bucle implacable de ensamblar contexto, instruir al modelo, observar el resultado y luego reensamblar un contexto para el siguiente paso. El contexto puede incluir instrucciones del sistema, entrada del usuario, historial de sesión, memorias a largo plazo, conocimiento de anclaje de fuentes autoritativas, qué herramientas podrían usarse y los resultados de herramientas ya invocadas. Esta sofisticada gestión de la atención del modelo permite que sus capacidades de razonamiento resuelvan para circunstancias novedosas y logren objetivos.

El Proceso de Resolución de Problemas de Agentes

Hemos definido un agente de IA como una aplicación completa y orientada a objetivos que integra un modelo de razonamiento, herramientas accionables y una capa de orquestación gobernante. Una versión corta es "LMs en un bucle con herramientas para lograr un objetivo".

Pero ¿cómo funciona realmente este sistema? ¿Qué hace un agente desde el momento en que recibe una solicitud hasta el momento en que entrega un resultado?

En su núcleo, un agente opera en un proceso continuo y cíclico para lograr sus objetivos. Si bien este bucle puede volverse altamente complejo, puede descomponerse en cinco pasos fundamentales como se discute en detalle en el libro *Agentic System Design*:6

1. Obtener la Misión: El proceso se inicia con un objetivo específico y de alto nivel. Esta misión es proporcionada por un usuario (por ejemplo, "Organiza el viaje de mi equipo para la próxima conferencia") o un disparador automatizado (por ejemplo, "Ha llegado un nuevo ticket de cliente de alta prioridad").
2. Explorar la Escena: El agente percibe su entorno para recopilar contexto. Esto implica que la capa de orquestación acceda a sus recursos disponibles: "¿Qué dice la solicitud del usuario?",

"¿Qué información hay en mi memoria a corto plazo? ¿Ya intenté hacer esta tarea? ¿Me dio el usuario orientación la semana pasada?", "¿Qué puedo acceder desde mis herramientas, como calendarios, bases de datos o APIs?"

3. Pensarlo Detalladamente: Este es el bucle central de "pensamiento" del agente, impulsado por el modelo de razonamiento. El agente analiza la Misión (Paso 1) contra la Escena (Paso 2) y diseña un plan. Esto no es un solo pensamiento, sino a menudo una cadena de razonamiento: "Para reservar viajes, primero necesito saber quién está en el equipo. Usaré la herramienta `get_team_roster`. Luego necesitaré verificar su disponibilidad a través de `calendar_api`."

4. Tomar Acción: La capa de orquestación ejecuta el primer paso concreto del plan. Selecciona e invoca la herramienta apropiada: llamando a una API, ejecutando una función de código o consultando una base de datos. Esto es el agente actuando en el mundo más allá de su propio razonamiento interno.

5. Observar e Iterar: El agente observa el resultado de su acción. La herramienta `get_team_roster` devuelve una lista de cinco nombres. Esta nueva información se agrega al contexto o "memoria" del agente. El bucle luego se repite, volviendo al Paso 3: "Ahora que tengo la lista, mi siguiente paso es verificar el calendario para estas cinco personas. Usaré `calendar_api`."

Este ciclo de "Pensar, Actuar, Observar" continúa - gestionado por la Capa de Orquestación, razonado por el Modelo y ejecutado por las Herramientas - hasta que el plan interno del agente está completo y la Misión inicial se logra.

Tomemos un ejemplo del mundo real de cómo operaría un Agente de Soporte al Cliente en este ciclo de 5 pasos:

Imagina que un usuario pregunta, "¿Dónde está mi pedido #12345?"

En lugar de actuar inmediatamente, el agente primero entra en su fase "Pensarlo Detalladamente" para diseñar una estrategia completa. Razona:

"El usuario quiere un estado de entrega. Para proporcionar una respuesta completa, necesito un plan de múltiples pasos:

1. Identificar: Primero debo encontrar el pedido en nuestra base de datos interna para confirmar que existe y obtener sus detalles.
2. Rastrear: De esos detalles, necesitaré extraer el número de seguimiento del transportista. Luego, consultaré la API externa del transportista para el estado en vivo.
3. Informar: Finalmente, debo sintetizar la información que he recopilado en una respuesta clara y útil para el usuario."

Con este plan de múltiples pasos en mente, el agente comienza la ejecución.

En su primera fase "Actuar", ejecuta el primer paso de su plan, llamando a la herramienta `find_order("12345")`. Observa el resultado: un registro completo del pedido, incluyendo el número de seguimiento "ZYX987."

La capa de orquestación del agente reconoce que la primera parte de su plan está completa y procede inmediatamente a la segunda. Actúa llamando a la herramienta `get_shipping_status("ZYX987")`. Observa el nuevo resultado: "En entrega."

Finalmente, habiendo ejecutado con éxito las etapas de recopilación de datos de su plan, el agente se mueve al paso "Informar". Percibe que tiene todos los componentes necesarios, planifica el mensaje final y actúa generando la respuesta: "¡Tu pedido #12345 está 'En entrega'!"

Una Taxonomía de Sistemas de Agentes

Comprender el bucle operativo de 5 pasos es la primera parte del rompecabezas. La segunda es reconocer que este bucle puede escalarse en complejidad para crear diferentes clases de agentes.

Para un arquitecto o líder de producto, una decisión inicial clave es definir qué tipo de agente construir. Podemos clasificar los sistemas de agentes en unos pocos niveles amplios, cada uno construyendo sobre las capacidades del anterior.

Nivel 0: El Sistema de Razonamiento Central

Antes de tener un agente, debemos comenzar con el "Cerebro" en su forma más básica: el motor de razonamiento en sí. En esta configuración, un Modelo de Lenguaje (LM) opera de forma aislada, respondiendo únicamente basado en su vasto conocimiento pre-entrenado sin herramientas, memoria o interacción con el entorno en vivo.

Su fortaleza reside en este extenso entrenamiento, permitiéndole explicar conceptos establecidos y planificar cómo abordar la resolución de un problema con gran profundidad. La desventaja es una falta total de conciencia en tiempo real; es funcionalmente "ciego" a cualquier evento o hecho fuera de sus datos de entrenamiento.

Por ejemplo, puede explicar las reglas del béisbol profesional y la historia completa de los New York Yankees. Pero si preguntas, "¿Cuál fue el marcador final del juego de los Yankees anoche?", no podría responder. Ese juego es un evento específico del mundo real que ocurrió después de que se recopilaran sus datos de entrenamiento, por lo que la información simplemente no existe en su conocimiento.

Nivel 1: El Solucionador de Problemas Conectado

En este nivel, el motor de razonamiento se convierte en un agente funcional al conectarse y utilizar herramientas externas: el componente "Manos" de nuestra arquitectura. Su resolución de problemas ya no está confinada a su conocimiento estático pre-entrenado.

Usando el bucle de 5 pasos, el agente ahora puede responder nuestra pregunta anterior. Dada la "Misión": "¿Cuál fue el marcador final del juego de los Yankees anoche?", su paso "Pensar" reconoce esto como una necesidad de datos en tiempo real. Su paso "Actuar" luego invoca una herramienta, como una API de Google Search con la fecha y términos de búsqueda apropiados. "Observa" el resultado de búsqueda (por ejemplo, "Yankees ganaron 5-3"), y sintetiza ese hecho en una respuesta final.

Esta capacidad fundamental de interactuar con el mundo: ya sea usando una herramienta de búsqueda para un marcador, una API financiera para un precio de acción en vivo, o una base de datos a través de Generación Aumentada por Recuperación (RAG) es la capacidad central de un agente de Nivel 1.

Nivel 2: El Solucionador de Problemas Estratégico

El Nivel 2 marca una expansión significativa en capacidad, pasando de ejecutar tareas simples a planificar estratégicamente objetivos complejos y de múltiples partes. La habilidad clave que emerge aquí es la ingeniería de contexto: la capacidad del agente de seleccionar activamente, empaquetar y gestionar la información más relevante para cada paso de su plan.

La precisión de un agente depende de un contexto enfocado y de alta calidad. La ingeniería de contexto cura la atención limitada del modelo para evitar sobrecarga y asegurar un rendimiento eficiente.

Por ejemplo, considera la "Misión": "Encuentra una buena cafetería a mitad de camino entre mi oficina en 1600 Amphitheatre Parkway, Mountain View, y la oficina de mi cliente en 1 Market St, San Francisco."

Un agente de Nivel 2 comenzará a crear un plan:

1. Pensar: "Primero debo encontrar el punto medio."
 - Actuar: Llamar a la herramienta Maps con ambas direcciones.
 - Observar: "El punto medio es Millbrae, CA."
2. Pensar: "Ahora debo encontrar cafeterías en Millbrae. El usuario pidió 'buenas', así que buscaré lugares con una calificación de 4 estrellas o superior."
 - Actuar: Llamar a la herramienta google_places con query="cafetería en Millbrae, CA", min_rating=4.0. (Esto es ingeniería de contexto: automáticamente creó una nueva consulta de búsqueda enfocada desde la salida del paso anterior).
 - Observar: "La búsqueda devuelve 'Millbrae Coffee' y 'The Daily Grind'."
3. Pensar: "Sintetizaré estos resultados y los presentaré al usuario."

Esta planificación estratégica también permite asistencia proactiva, como un agente que lee un largo correo electrónico de confirmación de vuelo, ingenieriza el contexto clave (número de vuelo, fecha) y actúa agregándolo a tu calendario.

Nivel 3: El Sistema Multiagente Colaborativo

En el nivel más alto, el paradigma cambia completamente. Nos alejamos de construir un único agente "súper poderoso" y hacia un "equipo de especialistas" trabajando en concierto, un modelo que refleja directamente una organización humana. La fuerza colectiva del sistema reside en esta división del trabajo.

Aquí, los agentes tratan a otros agentes como herramientas. Imagina un agente "Project Manager" recibiendo una "Misión": "Lanzar nuestros nuevos auriculares 'Solaris'."

El agente Project Manager no hace todo el trabajo por sí mismo. Actúa creando nuevas Misiones para su equipo de agentes especializados, muy similar a como funciona en la vida real:

1. Delega a MarketResearchAgent: "Analiza los precios de la competencia para auriculares con cancelación de ruido. Devuelve un documento resumen para mañana."
2. Delega a MarketingAgent: "Redacta tres versiones de un comunicado de prensa usando la hoja de especificaciones del producto 'Solaris' como contexto."
3. Delega a WebDevAgent: "Genera el HTML de la nueva página de producto basado en los maquetos de diseño adjuntos."

Este modelo colaborativo, aunque actualmente limitado por las limitaciones de razonamiento de los LMs de hoy, representa la frontera de automatizar flujos de trabajo empresariales complejos de principio a fin.

Nivel 4: El Sistema Autoevolutivo

El Nivel 4 representa un salto profundo de la delegación a la creación y adaptación autónoma. En este nivel, un sistema de agentes puede identificar brechas en sus propias capacidades y crear dinámicamente nuevas herramientas o incluso nuevos agentes para llenarlas. Pasa de usar un conjunto fijo de recursos a expandirlos activamente.

Siguiendo nuestro ejemplo, el agente "Project Manager", encargado del lanzamiento de 'Solaris', podría darse cuenta de que necesita monitorear el sentimiento en redes sociales, pero no existe tal herramienta o agente en su equipo.

1. Pensar (Meta-Razonamiento): "Debo rastrear el zumbido en redes sociales para 'Solaris', pero me falta la capacidad."
2. Actuar (Creación Autónoma): En lugar de fallar, invoca una herramienta AgentCreator de alto nivel con una nueva misión: "Construye un nuevo agente que monitorea redes sociales para las

palabras clave 'auriculares Solaris', realiza análisis de sentimiento e informa un resumen diario."

3. Observar: Se crea, prueba y agrega un nuevo agente especializado SentimentAnalysisAgent sobre la marcha, listo para contribuir a la misión original.

Este nivel de autonomía, donde un sistema puede expandir dinámicamente sus propias capacidades, convierte a un equipo de agentes en una organización verdaderamente aprendiente y evolutiva.

Arquitectura Central de Agentes: Modelo, Herramientas y Orquestación

Sabemos qué hace un agente y cómo puede escalar. ¿Pero cómo lo construimos realmente?

La transición del concepto al código reside en el diseño arquitectónico específico de sus tres componentes centrales.

Modelo: El "Cerebro" de tu Agente de IA

El LM es el núcleo de razonamiento de tu agente, y su selección es una decisión arquitectónica crítica que dicta las capacidades cognitivas de tu agente, el costo operativo y la velocidad.

Sin embargo, tratar esta elección como un simple asunto de elegir el modelo con el puntaje de referencia más alto es un camino común hacia el fracaso. El éxito de un agente en un entorno de producción rara vez se determina por referencias académicas genéricas.

El éxito en el mundo real exige un modelo que destaque en los fundamentos de agentes: razonamiento superior para navegar problemas complejos y de múltiples pasos y uso confiable de herramientas para interactuar con el mundo⁷.

Para hacer esto bien, comienza por definir el problema empresarial, luego prueba modelos contra métricas que se mapeen directamente a ese resultado. Si tu agente necesita escribir código, Pruébalo en tu base de código privada. Si procesa reclamaciones de seguros, evalúa su capacidad para extraer información de tus formatos de documento específicos. Este análisis debe luego referenciarse cruzadamente con las practicalidades del costo y la latencia. El "mejor" modelo es el que se encuentra en la intersección óptima de calidad, velocidad y precio para tu tarea específica⁸.

Puedes elegir más de un modelo, un "equipo de especialistas". No usas un martillo para partir una nuez. Una arquitectura de agente robusta podría usar un modelo fronterizo como Gemini 2.5 Pro para el trabajo pesado de planificación inicial y razonamiento complejo, pero luego enrutar inteligentemente tareas más simples y de alto volumen, como clasificar la intención del usuario o resumir texto, a un modelo mucho más rápido y rentable como Gemini 2.5 Flash. El enrutamiento de modelos puede ser automático o codificado pero es una estrategia clave para optimizar tanto el rendimiento como el costo⁹.

El mismo principio se aplica al manejo de diversos tipos de datos. Si bien un modelo multimodal nativo como Gemini live mode¹⁰ ofrece una ruta simplificada para procesar imágenes y audio, una alternativa es usar herramientas especializadas como la API de Cloud Vision¹¹ o la API de Speech-to-Text¹². En este patrón, el mundo primero se convierte a texto, que luego se pasa a un modelo de solo lenguaje para razonamiento. Esto añade flexibilidad y permite componentes de la mejor categoría, pero también introduce una complejidad significativa.

Finalmente, el panorama de la IA está en un estado de evolución constante y rápida. El modelo que elijas hoy será superado en seis meses. Una mentalidad de "configurar y olvidar" es insostenible. Construir para esta realidad significa invertir en un marco operativo ágil: una práctica "Agent Ops"¹³. Con una canalización CI/CD robusta que evalúa continuamente nuevos modelos contra tus métricas comerciales clave, puedes desmitificar y acelerar las actualizaciones, asegurando que tu agente siempre esté impulsado por el mejor cerebro disponible sin requerir una revisión arquitectónica completa.

Herramientas: Las "Manos" de tu Agente de IA

Si el modelo es el cerebro del agente, las herramientas son las manos que conectan su razonamiento con la realidad. Permiten que el agente vaya más allá de sus datos de entrenamiento estáticos para recuperar información en tiempo real y tomar acciones en el mundo. Una interfaz de herramienta robusta es un bucle de tres partes: definir qué puede hacer una herramienta, invocarla y observar el resultado.

Aquí hay algunos de los principales tipos de herramientas que los constructores de agentes pondrán en las "manos" de sus agentes. Para una inmersión más completa, consulta el documento blanco centrado en herramientas de agentes de esta serie.

Recuperación de Información: Anclaje en la Realidad

La herramienta más fundamental es la capacidad de acceder a información actualizada. La Generación Aumentada por Recuperación (RAG) le da al agente una "tarjeta de biblioteca" para consultar conocimiento externo, a menudo almacenado en Bases de Datos Vectoriales o Grafos de Conocimiento, que van desde documentos internos de la empresa hasta conocimiento web a través de Google Search. Para datos estructurados, las herramientas de Lenguaje Natural a SQL (NL2SQL) permiten que el agente consulte bases de datos para responder preguntas analíticas como, "¿Cuáles fueron nuestros productos más vendidos el último trimestre?" Al buscar antes de hablar, ya sea en un documento o una base de datos, el agente se ancla en hechos, reduciendo dramáticamente las alucinaciones.

Ejecución de Acciones: Cambiando el Mundo

El verdadero poder de los agentes se libera cuando pasan de leer información a hacer activamente cosas. Al envolver APIs existentes y funciones de código como herramientas, un

agente puede enviar un correo electrónico, programar una reunión o actualizar un registro de cliente en ServiceNow. Para tareas más dinámicas, un agente puede escribir y ejecutar código sobre la marcha. En un sandbox seguro, puede generar una consulta SQL o un script Python para resolver un problema complejo o realizar un cálculo, transformándolo de un asistente informado a un actor autónomo¹⁴.

Esto también incluye herramientas para interacción humana. Un agente puede usar una herramienta Humano en el Bucle (HITL) para pausar su flujo de trabajo y pedir confirmación (por ejemplo, `ask_for_confirmation()`) o solicitar información específica de una interfaz de usuario (por ejemplo, `ask_for_date_input()`), asegurando que una persona esté involucrada en decisiones críticas. HITL podría implementarse a través de mensajes SMS y una tarea en una base de datos.

Llamada a Funciones: Conectando Herramientas a tu Agente

Para que un agente pueda hacer "llamada a funciones" de manera confiable y usar herramientas, necesita instrucciones claras, conexiones seguras y orquestación¹⁵. Estándares de larga data como la especificación OpenAPI proporcionan esto, dando al agente un contrato estructurado que describe el propósito de una herramienta, sus parámetros requeridos y su respuesta esperada. Este esquema permite que el modelo genere la llamada de función correcta cada vez e interprete la respuesta de la API. Para descubrimiento y conexión más simples a herramientas, estándares abiertos como el Protocolo de Contexto de Modelo (MCP) se han vuelto populares porque son más convenientes¹⁶. Además, algunos modelos tienen herramientas nativas, como Gemini con Google Search nativo, donde la invocación de la función ocurre como parte de la llamada LM misma¹⁷.

La Capa de Orquestación

Si el modelo es el cerebro del agente y las herramientas son sus manos, la capa de orquestación es el sistema nervioso central que los conecta. Es el motor que ejecuta el bucle "Pensar, Actuar, Observar", la máquina de estados que gobierna el comportamiento del agente y el lugar donde la lógica cuidadosamente elaborada por el desarrollador cobra vida. Esta capa no es solo plomería; es el director de toda la sinfonía de agentes, decidiendo cuándo el modelo debe razonar, qué herramienta debe actuar y cómo los resultados de esa acción deben informar el siguiente movimiento.

Decisiones de Diseño Central

La primera decisión arquitectónica es determinar el grado de autonomía del agente. La elección existe en un espectro. En un extremo, tienes flujos de trabajo deterministas y predecibles que llaman a un LM como herramienta para una tarea específica: un toque de IA para aumentar un proceso existente. En el otro extremo, tienes el LM en el asiento del conductor, adaptándose dinámicamente, planificando y ejecutando tareas para lograr un objetivo.

Una elección paralela es el método de implementación. Los constructores sin código ofrecen velocidad y accesibilidad, empoderando a los usuarios empresariales para automatizar tareas estructuradas y construir agentes simples rápidamente. Para sistemas más complejos y críticos para la misión, los marcos de código primero, como el Kit de Desarrollo de Agentes (ADK) de Google¹⁸, proporcionan el control profundo, la personalización y las capacidades de integración que los ingenieros requieren.

Independientemente del enfoque, un marco de nivel de producción es esencial. Debe ser abierto, permitiéndote conectar cualquier modelo o herramienta para evitar el bloqueo del proveedor. Debe proporcionar control preciso, permitiendo un enfoque híbrido donde el razonamiento no determinista de un LM es gobernado por reglas comerciales codificadas. Lo más importante, el marco debe estar construido para observabilidad. Cuando un agente se comporta de manera inesperada, no puedes simplemente poner un punto de interrupción en el "pensamiento" del modelo. Un marco robusto genera trazas y registros detallados, exponiendo toda la trayectoria de razonamiento: el monólogo interno del modelo, la herramienta que eligió, los parámetros que generó y el resultado que observó.

Instruir con Conocimiento del Dominio y Persona

Dentro de este marco, la palanca más poderosa del desarrollador es instruir al agente con conocimiento del dominio y una persona distinta. Esto se logra a través de una instrucción del sistema o un conjunto de instrucciones centrales. Esto no es solo un comando simple; es la constitución del agente.

Aquí, le dices, "Eres un agente de soporte al cliente servicial para Acme Corp, ..." y proporcionas restricciones, esquema de salida deseado, reglas de compromiso, un tono de voz específico, y orientación explícita sobre cuándo y por qué debería usar sus herramientas. Algunos escenarios de ejemplo en las instrucciones suelen ser muy efectivos.

Aumentar con Contexto

La "memoria" del agente se orquesta en la ventana de contexto del LM en tiempo de ejecución. Para una inmersión más completa, consulta el documento blanco centrado en memoria de agentes de esta serie.

La memoria a corto plazo es el "bloc de notas" activo del agente, manteniendo el historial en ejecución de la conversación actual. Rastrea la secuencia de pares (Acción, Observación) del bucle en curso, proporcionando el contexto inmediato que el modelo necesita para decidir qué hacer a continuación. Esto puede implementarse como abstracciones como estado, artefactos, sesiones o hilos.

La memoria a largo plazo proporciona persistencia a través de sesiones. Arquitectónicamente, esto casi siempre se implementa como otra herramienta especializada: un sistema RAG conectado a una base de datos vectorial o motor de búsqueda. El orquestador le da al agente

la capacidad de prebuscar y consultar activamente su propio historial, permitiéndole "recordar" las preferencias de un usuario o el resultado de una tarea similar de hace semanas para una experiencia verdaderamente personalizada y continua¹⁹.

Sistemas Multiagente y Patrones de Diseño

A medida que las tareas crecen en complejidad, construir un único agente "súper poderoso" se vuelve ineficiente. La solución más efectiva es adoptar un enfoque de "equipo de especialistas", que refleja una organización humana. Este es el núcleo de un sistema multiagente: un proceso complejo se segmenta en subtareas discretas, y cada una se asigna a un agente de IA especializado y dedicado.

Esta división del trabajo permite que cada agente sea más simple, más enfocado y más fácil de construir, probar y mantener, lo cual es ideal para procesos empresariales dinámicos o de larga duración.

Los arquitectos pueden confiar en patrones de diseño de agentes probados, aunque las capacidades de los agentes y por lo tanto los patrones están evolucionando rápidamente²⁰. Para tareas dinámicas o no lineales, el patrón Coordinador es esencial. Introduce un agente "gerente" que analiza una solicitud compleja, segmenta la tarea principal y enruta inteligentemente cada subtask al agente especializado apropiado (como un investigador, un escritor o un programador). El coordinador luego agrega las respuestas de cada especialista para formular una respuesta final y comprensiva.

Para flujos de trabajo más lineales, el patrón Secuencial es más adecuado, actuando como una línea de ensamblaje digital donde la salida de un agente se convierte en la entrada directa para el siguiente. Otros patrones clave se centran en la calidad y seguridad. El patrón de Refinamiento Iterativo crea un bucle de retroalimentación, usando un agente "generador" para crear contenido y un agente "crítico" para evaluarlo contra estándares de calidad. Para tareas de alto riesgo, el patrón Humano en el Bucle (HITL) es crítico, creando una pausa deliberada en el flujo de trabajo para obtener aprobación de una persona antes de que un agente tome una acción significativa.

Despliegue de Agentes y Servicios

Después de haber construido un agente local, querrás desplegarlo en un servidor donde se ejecute todo el tiempo y donde otras personas y agentes puedan usarlo. Continuando nuestra analogía, el despliegue y los servicios serían el cuerpo y las piernas para nuestro agente. Un agente requiere varios servicios para ser efectivo, historial de sesiones y persistencia de memoria, y más. Como constructor de agentes, también serás responsable de decidir qué registras, y qué medidas de seguridad tomas para la privacidad de datos y cumplimiento de regulaciones de residencia de datos. Todos estos servicios están dentro del alcance al desplegar agentes en producción.

Afortunadamente, los constructores de agentes pueden confiar en décadas de infraestructura de alojamiento de aplicaciones. Los agentes son una nueva forma de software después de todo y muchos de los mismos principios aplican. Los constructores pueden confiar en opciones de despliegue específicas para agentes como Vertex AI Agent Engine que soportan el tiempo de ejecución y todo lo demás en una plataforma²¹. Para desarrolladores de software que quieren controlar sus pilas de aplicaciones más directamente, o desplegar agentes dentro de su infraestructura DevOps existente, cualquier agente y la mayoría de los servicios de agentes pueden agregarse a un contenedor docker y desplegarse en tiempos de ejecución estándar de la industria como Cloud Run o GKE²².

Si no eres un desarrollador de software y un experto en DevOps, el proceso de desplegar tu primer agente puede ser abrumador. Muchos marcos de agentes facilitan esto con un comando de despliegue o una plataforma dedicada para desplegar el agente, y estos deberían usarse para exploración e incorporación tempranas. Escalar a un entorno seguro y listo para producción generalmente requerirá una mayor inversión de tiempo y aplicación de mejores prácticas, incluyendo CI/CD y pruebas automatizadas para tus agentes²³.

Agent Ops: Un Enfoque Estructurado para lo Impredecible

A medida que construyes tus primeros agentes, estarás probando manualmente el comportamiento una y otra vez. ¿Cuando agregas una función, funciona? ¿Cuando arreglas un error, causaste un problema diferente? Las pruebas son normales para el desarrollo de software pero funcionan de manera diferente con la IA generativa.

La transición del software tradicional y determinista a los sistemas estocásticos de agentes requiere una nueva filosofía operativa. Las pruebas unitarias de software tradicional simplemente podían afirmar salida == esperado; pero eso no funciona cuando la respuesta de un agente es probabilística por diseño. Además, porque el lenguaje es complicado, generalmente requiere un LM para evaluar la "calidad": que la respuesta del agente hace todo lo que debería, nada que no debería, y con el tono adecuado.

Agent Ops es el enfoque disciplinado y estructurado para gestionar esta nueva realidad. Es una evolución natural de DevOps y MLOps, adaptada para los desafíos únicos de construir, desplegar y gobernar agentes de IA, convirtiendo la imprevisibilidad de un pasivo en una característica gestionada, medible y confiable²⁴. Para una inmersión más completa, consulta el documento blanco centrado en calidad de agentes de esta serie.

Medir lo que Importa: Instrumentación del Éxito como un Experimento A/B

Antes de poder mejorar tu agente, debes definir qué significa "mejor" en el contexto de tu negocio. Enmarca tu estrategia de observabilidad como una prueba A/B y pregúntate: ¿cuáles son los Indicadores Clave de Rendimiento (KPIs) que demuestran que el agente está entregando valor? Estas métricas deberían ir más allá de la corrección técnica y medir el impacto en el mundo real: tasas de finalización de objetivos, puntuaciones de satisfacción del

usuario, latencia de tareas, costo operativo por interacción y, lo más importante, el impacto en objetivos comerciales como ingresos, conversión o retención de clientes.

Esta vista de arriba hacia abajo guiará el resto de tus pruebas, te pone en el camino del desarrollo impulsado por métricas y te permitirá calcular un retorno de inversión.

Calidad en lugar de Aprobado/Reprobado: Usando un Juez LM

Las métricas comerciales no te dicen si el agente se está comportando correctamente. Dado que un simple aprobado/reprobado es imposible, cambiamos a evaluar por calidad usando un "LM como Juez". Esto implica usar un modelo poderoso para evaluar la salida del agente contra una rúbrica predefinida: ¿Dio la respuesta correcta? ¿Estaba la respuesta anclada en hechos? ¿Siguió las instrucciones? Esta evaluación automatizada, ejecutada contra un conjunto de datos dorado de instrucciones, proporciona una medida consistente de calidad.

Crear los conjuntos de datos de evaluación, que incluyen las preguntas ideales (o "doradas") y respuestas correctas, puede ser un proceso tedioso. Para construir estos, deberías muestrear escenarios de interacciones de producción o desarrollo existentes con el agente. El conjunto de datos debe cubrir toda la amplitud de casos de uso que esperas que tus usuarios involucren, además de algunos inesperados. Si bien la inversión en evaluación se paga rápidamente, los resultados de evaluación siempre deben ser revisados por un experto en el dominio antes de ser aceptados como válidos. Cada vez más, la curación y mantenimiento de estas evaluaciones se está convirtiendo en una responsabilidad clave para los Gerentes de Producto con el apoyo de expertos en el dominio.

Desarrollo Impulsado por Métricas: Tu Go/No-Go para el Despliegue

Una vez que has automatizado docenas de escenarios de evaluación y establecido puntuaciones de calidad confiables, puedes probar con confianza los cambios en tu agente de desarrollo. El proceso es simple: ejecuta la nueva versión contra todo el conjunto de datos de evaluación, y compara directamente sus puntuaciones con la versión de producción existente. Este sistema robusto elimina las conjeturas, asegurando que estés confiado en cada despliegue. Si bien las evaluaciones automatizadas son críticas, no olvides otros factores importantes como latencia, costo y tasas de éxito de tareas. Para máxima seguridad, usa despliegues A/B para desplegar lentamente nuevas versiones y comparar estas métricas de producción del mundo real junto con tus puntuaciones de simulación.

Depurar con Trazas de OpenTelemetry: Respondiendo "¿Por qué?"

Cuando tus métricas caen o un usuario reporta un error, necesitas entender "por qué". Una traza de OpenTelemetry es un registro de alta fidelidad, paso a paso, de toda la ruta de ejecución del agente (trayectoria), permitiéndote depurar los pasos del agente²⁵. Con trazas, puedes ver la instrucción exacta enviada al modelo, el razonamiento interno del modelo (si está disponible), la herramienta específica que eligió llamar, los parámetros precisos que generó

para esa herramienta, y los datos brutos que volvieron como observación. Las trazas pueden ser complicadas la primera vez que las miras pero proporcionan los detalles necesarios para diagnosticar y arreglar la causa raíz de cualquier problema. Los detalles importantes de traza pueden convertirse en métricas, pero revisar trazas es principalmente para depuración, no para resúmenes de rendimiento. Los datos de traza pueden recopilarse perfectamente en plataformas como Google Cloud Trace, que visualizan y buscan a través de vastas cantidades de trazas, agilizando el análisis de causa raíz.

Valorar los Comentarios Humanos: Guiando tu Automatización

Los comentarios humanos no son una molestia con la que lidiar; son el recurso más valioso y rico en datos que tienes para mejorar tu agente. Cuando un usuario reporta un error o hace clic en el botón "pulgar abajo", te están dando un regalo: un nuevo caso límite del mundo real que tus escenarios de evaluación automatizados pasaron por alto. Recopilar y agregar estos datos es crítico; cuando ves un número estadísticamente significativo de informes similares o caídas de métricas, debes vincular las ocurrencias de vuelta a tu plataforma de análisis para generar información y activar alertas para problemas operativos. Un proceso efectivo de Agent Ops "cierra el bucle" capturando estos comentarios, replicando el problema y convirtiendo ese escenario específico en un nuevo caso de prueba permanente en tu conjunto de datos de evaluación. Esto asegura que no solo arreglas el error sino que también vacunas al sistema contra que toda esa clase de error vuelva a ocurrir.

Interoperabilidad de Agentes

Una vez que construyes tus agentes de alta calidad, quieres poder interconectarlos con usuarios y otros agentes. En nuestra analogía de partes del cuerpo, esta sería la cara del Agente. Hay una diferencia entre conectar a agentes versus conectar agentes con datos y APIs; Los agentes no son herramientas²⁶. Supongamos que ya tienes herramientas conectadas a tus agentes, ahora consideremos cómo traes tus agentes a un ecosistema más amplio.

Agentes y Humanos

La forma más común de interacción agente-humano es a través de una interfaz de usuario. En su forma más simple, esto es un chatbot, donde un usuario escribe una solicitud y el agente, actuando como un servicio backend, la procesa y devuelve un bloque de texto. Los agentes más avanzados pueden proporcionar datos estructurados, como JSON, para potenciar experiencias front-end ricas y dinámicas. Los patrones de interacción Humano en el Bucle (HITL) incluyen refinamiento de intención, expansión de objetivos, confirmación y solicitudes de aclaración.

El uso de computadora es una categoría de herramienta donde el LM toma el control de una interfaz de usuario, a menudo con interacción y supervisión humana. Un agente habilitado para uso de computadora puede decidir que la siguiente mejor acción es navegar a una nueva

página, resaltar un botón específico o rellenar previamente un formulario con información relevante²⁷.

En lugar de un agente usando una interfaz en nombre del usuario, el LM puede cambiar la UI para satisfacer las necesidades del momento. Esto puede hacerse con Herramientas que controlan UI (MCP UI)²⁸, o sistemas de mensajería UI especializados que pueden sincronizar el estado del cliente con un agente (AG UI)²⁹, e incluso generación de interfaces personalizadas (A2UI)³⁰.

Por supuesto, la interacción humana no se limita a pantallas y teclados. Los agentes avanzados están rompiendo la barrera del texto y moviéndose hacia la comunicación multimodal en tiempo real con "modo en vivo" creando una conexión más natural y similar a la humana. Tecnologías como la Gemini Live API³¹ habilitan el streaming bidireccional, permitiendo que un usuario hable con un agente y lo interrumpa, tal como lo harían en una conversación natural.

Esta capacidad cambia fundamentalmente la naturaleza de la colaboración agente-humano. Con acceso a la cámara y micrófono de un dispositivo, el agente puede ver lo que el usuario ve y escuchar lo que dicen, respondiendo con voz generada a una latencia que imita la conversación humana.

Esto abre una vasta gama de casos de uso que son simplemente imposibles con texto, desde un técnico recibiendo orientación manos libres mientras repara un equipo hasta un comprador obteniendo consejos de estilo en tiempo real. Hace que el agente sea un socio más intuitivo y accesible.

Agentes y Agentes

Así como los agentes deben conectarse con humanos, también deben conectarse entre sí. A medida que una empresa escala su uso de IA, diferentes equipos construirán diferentes agentes especializados. Sin un estándar común, conectarlos requeriría construir una enredadera de integraciones de API personalizadas y frágiles que son imposibles de mantener. El desafío principal es doble: descubrimiento (¿cómo encuentra mi agente otros agentes y sabe qué pueden hacer?) y comunicación (¿cómo nos aseguramos de que hablen el mismo idioma?).

El protocolo Agente2Agente (A2A) es el estándar abierto diseñado para resolver este problema. Actúa como un apretón de manos universal para la economía de agentes. A2A permite que cualquier agente publique una "tarjeta de presentación" digital, conocida como Tarjeta de Agente. Este simple archivo JSON anuncia las capacidades del agente, su punto final de red y las credenciales de seguridad requeridas para interactuar con él.

Esto hace que el descubrimiento sea simple y estandarizado. A diferencia de MCP que se enfoca en resolver solicitudes transaccionales, la comunicación Agente a Agente es típicamente para resolución adicional de problemas.

Una vez descubiertos, los agentes se comunican usando una arquitectura orientada a tareas. En lugar de una simple solicitud-respuesta, las interacciones se enmarcan como "tareas" asíncronas. Un agente cliente envía una solicitud de tarea a un agente servidor, que luego puede proporcionar actualizaciones de streaming mientras trabaja en el problema a través de una conexión de larga duración. Este protocolo de comunicación robusto y estandarizado es la pieza final del rompecabezas, habilitando los sistemas multiagente colaborativos de Nivel 3 que representan la frontera de la automatización. A2A transforma una colección de agentes aislados en un ecosistema verdaderamente interoperable.

Agentes y Dinero

A medida que los agentes de IA hacen más tareas por nosotros, algunas de esas tareas implican comprar o vender, negociar o facilitar transacciones. La web actual está construida para humanos haciendo clic en "comprar", la responsabilidad está en el humano. Si un agente autónomo hace clic en "comprar" crea una crisis de confianza: si algo sale mal, ¿quién tiene la culpa? Estos son problemas complejos de autorización, autenticidad y rendición de cuentas. Para desbloquear una verdadera economía de agentes, necesitamos nuevos estándares que permitan a los agentes transactar de manera segura y confiable en nombre de sus usuarios.

Esta área emergente está lejos de establecerse, pero dos protocolos clave están allanando el camino. El Protocolo de Pagos de Agentes (AP2) es un protocolo abierto diseñado para ser el lenguaje definitivo para el comercio de agentes. Extiende protocolos como A2A introduciendo "mandatos" firmados criptográficamente. Estos actúan como prueba verificable de intención del usuario, creando una pista de auditoría no repudiable para cada transacción. Esto permite que un agente navegue, negocie y transacte a escala global basado en autoridad delegada del usuario. Complementando esto está x402, un protocolo de pago de internet abierto que usa el código de estado HTTP 402 estándar "Pago Requerido". Habilita micropagos máquina a máquina sin fricción, permitiendo que un agente pague por cosas como acceso a API o contenido digital en base a uso sin necesitar cuentas complejas o suscripciones. Juntos, estos protocolos están construyendo la capa de confianza fundamental para la web de agentes.

Asegurar un Agente Individual: El Compromiso de Confianza

Cuando creas tu primer agente de IA, inmediatamente enfrentas una tensión fundamental: el compromiso entre utilidad y seguridad. Para hacer que un agente sea útil, debes darle poder: la autonomía para tomar decisiones y las herramientas para realizar acciones como enviar correos electrónicos o consultar bases de datos. Sin embargo, cada onza de poder que otorgas introduce una medida correspondiente de riesgo. Las principales preocupaciones de seguridad son acciones no autorizadas: comportamientos no deseados o dañinos y divulgación de datos sensibles. Quieres darle a tu agente una correa lo suficientemente larga para hacer su trabajo,

pero lo suficientemente corta para evitar que se meta en tráfico, especialmente cuando ese tráfico involucra acciones irreversibles o los datos privados de tu empresa³².

Para gestionar esto, no puedes confiar únicamente en el juicio del modelo de IA, ya que puede ser manipulado por técnicas como inyección de instrucciones³³. En cambio, la mejor práctica es un enfoque híbrido de defensa en profundidad³⁴. La primera capa consiste en barreras deterministas tradicionales: un conjunto de reglas codificadas que actúan como un punto de estrangulamiento de seguridad fuera del razonamiento del modelo. Esto podría ser un motor de políticas que bloquea cualquier compra por encima de \$100 o requiere confirmación explícita del usuario antes de que el agente pueda interactuar con una API externa. Esta capa proporciona límites duros predecibles y auditables sobre el poder del agente.

La segunda capa aprovecha las defensas basadas en razonamiento, usando IA para ayudar a asegurar la IA. Esto implica entrenar el modelo para que sea más resistente a ataques (entrenamiento adversarial) y emplear modelos "guardianes" más pequeños y especializados que actúan como analistas de seguridad. Estos modelos pueden examinar el plan propuesto por el agente antes de que se ejecute, marcando pasos potencialmente riesgosos o que violan políticas para su revisión. Este modelo híbrido, combinando la certeza rígida del código con la conciencia contextual de la IA, crea una postura de seguridad robusta incluso para un solo agente, asegurando que su poder siempre esté alineado con su propósito.

Identidad del Agente: Una Nueva Clase de Principal

En el modelo de seguridad tradicional, hay usuarios humanos que podrían usar OAuth o SSO, y servicios que usan IAM o cuentas de servicio. Los agentes añaden una 3ª categoría de principal. Un agente no es meramente un pieza de código; es un actor autónomo, un nuevo tipo de principal que requiere su propia identidad verificable. Así como a los empleados se les emite una identificación, cada agente en la plataforma debe emitir un "pasaporte digital" seguro y verificable. Esta Identidad de Agente es distinta de la identidad del usuario que lo invocó y del desarrollador que lo construyó. Este es un cambio fundamental en cómo debemos abordar la Gestión de Identidad y Acceso (IAM) en la empresa.

Hacer que cada identidad sea verificada y tener controles de acceso para todas ellas es la base de la seguridad de agentes. Una vez que un agente tiene una identidad verificable criptográficamente (a menudo usando estándares como SPIFFE³⁵), se le pueden otorgar permisos específicos de privilegio mínimo. El SalesAgent se otorga acceso de lectura/escritura al CRM, mientras que el HRonboardingAgent se le niega explícitamente. Este control granular...

Dia # 2

Agent Tools & Interoperability with MCP

****Autores:**** Mike Styer, Kanchana Patlolla, Madhuranjan Mohan y Sal Diaz

****Agent Tools & Interoperability with MCP****

****Noviembre de 2025****

Agradecimientos

****Colaboradores de contenido****

- Antony Arul
- Ruben Gonzalez
- Che Liu
- Kimberly Milam
- Anant Nawalgaria
- Geir Sjurseth

****Curadores y editores****

- Anant Nawalgaria
- Kanchana Patlolla

****Diseñador****

- Michael Lanning

Tabla de contenidos

Introducción: Modelos, Herramientas y Agentes 7

Herramientas y llamada a funciones 8

¿Qué entendemos por herramienta? 8

Tipos de herramientas 10

Herramientas integradas 11

Herramientas de Agente 13

Mejores Prácticas 15

La documentación es importante 15

Describir acciones, no implementaciones 17

Publicar tareas, no llamadas a la API 18

Haz las herramientas lo más granulares posible 18

Diseña para una salida concisa 19

Usa la validación de manera efectiva 19

Entendiendo el Protocolo de Contexto de Modelo 20

El Problema de Integración "N x M" y la necesidad de Estandarización 20

Componentes Arquitectónicos Centrales: Hosts, Clientes y Servidores	21
La Capa de Comunicación: JSON-RPC, Transportes y Tipos de Mensaje	22
Primitivas Clave: Herramientas y otras	24
Definición de Herramienta	26
Resultados de Herramienta	28
Contenido Estructurado	29
Manejo de Errores	29
Otras Capacidades	31
Recursos	31
Instrucciones	31
Muestreo	32
Elicitación	33
Raíces	33
Protocolo de Contexto de Modelo: A Favor y en Contra	34
Capacidades y Ventajas Estratégicas	34
Aceleración del Desarrollo y Fomento de un Ecosistema Reutilizable	34
Tabla de contenidos	
Flexibilidad Arquitectónica y Futuro-Prueba	35
Fundamentos para la Gobernanza y el Control	36
Riesgos Críticos y Desafíos	36
Brechas de Preparación Empresarial	38
Seguridad en MCP	39
Nuevo panorama de amenazas	39
Riesgos y Mitigaciones	40
Suplantación de Herramientas	42
Definiciones de Herramientas Maliciosas y Contenidos Consumidos	44
Fugas de Información Sensible	45
Sin soporte para limitar el alcance del acceso	46
Conclusión	48
Apéndice	49
El problema del Confused Deputy	49
El Escenario: Un Repositorio de Código Corporativo	49
El Ataque	50
Tabla de contenidos	
El Resultado	51
Notas Finales	52

Introducción: Modelos, Herramientas y Agentes

Sin acceso a funciones externas, incluso el modelo fundamental más avanzado es solo un motor de predicción de patrones. Un modelo avanzado puede hacer muchas cosas bien: aprobar exámenes de leyes², escribir código³ o poesía⁴, crear imágenes⁵ y videos⁶, resolver problemas matemáticos⁷, pero por sí solo solo puede generar contenido basado en los datos con los que fue entrenado previamente. No puede acceder a ningún dato nuevo sobre el mundo excepto lo que se le introduce en su contexto de solicitud; no puede interactuar con un sistema externo; y no puede tomar ninguna acción para influir en su entorno.

La mayoría de los modelos fundamentales modernos ahora tienen la capacidad de llamar a funciones externas, o herramientas, para abordar esta limitación. Como las aplicaciones en un smartphone, las herramientas permiten que un sistema de IA haga más que solo generar patrones. Estas herramientas actúan como los "ojos" y las "manos" del agente, permitiéndole percibir y actuar sobre el mundo.

Unificando Agentes, Herramientas y el Mundo

Con la llegada de la IA de Agentes (Agentic AI), las herramientas se vuelven aún más importantes para los sistemas de IA. Un Agente de IA utiliza la capacidad de razonamiento de un modelo fundamental para interactuar con los usuarios y lograr objetivos específicos para ellos, y las herramientas externas le dan esa capacidad. Con la capacidad de tomar acciones externas, los agentes pueden tener un impacto dramático en las aplicaciones empresariales⁸.

Sin embargo, conectar herramientas externas a modelos fundamentales conlleva desafíos significativos, tanto problemas técnicos básicos como importantes riesgos de seguridad. El Protocolo de Contexto de Modelo⁹ se introdujo en 2024 como una forma de agilizar el proceso de integración de herramientas y modelos, y abordar algunos de estos desafíos técnicos y de seguridad.

En este documento hablamos primero sobre la naturaleza de las herramientas utilizadas por los modelos fundamentales: qué son y cómo usarlas. Damos algunas mejores prácticas y pautas para diseñar herramientas efectivas y usarlas de manera efectiva. Luego examinamos el Protocolo de Contexto de Modelo, hablando de sus componentes básicos y algunos de los desafíos y riesgos que conlleva. Finalmente, profundizamos en los desafíos de seguridad que plantea MCP a medida que se introduce en un entorno empresarial y se conecta a sistemas externos de alto valor.

Herramientas y llamada a funciones

¿Qué entendemos por herramienta?

En el mundo de la IA moderna, una herramienta es una función o un programa que una aplicación basada en LLM puede usar para realizar una tarea fuera de las capacidades del

modelo. El modelo en sí genera contenido para responder a la pregunta del usuario; las herramientas permiten que la aplicación interactúe con otros sistemas. En términos generales, las herramientas se ajustan a 2 tipos: permiten que un modelo sepa algo o que haga algo. En otras palabras, las herramientas pueden recuperar datos para que el modelo los use en solicitudes posteriores, accediendo a fuentes de datos estructuradas y no estructuradas; o, las herramientas pueden realizar una acción en nombre del usuario, a menudo llamando a una API externa o ejecutando otro código o función.

Un ejemplo de aplicación de una herramienta para un agente podría incluir llamar a una API para obtener el pronóstico del tiempo de la ubicación del usuario y presentar la información en las unidades preferidas del usuario. Esta es una pregunta simple, pero para responderla correctamente, el modelo necesitaría información sobre la ubicación actual del usuario y el clima actual; ninguno de estos datos está incluido en los datos de entrenamiento del modelo. El modelo también necesita poder convertir entre unidades de temperatura; si bien los modelos fundamentales están mejorando en sus capacidades matemáticas, este no es su punto fuerte y los cálculos matemáticos son otro área donde generalmente es mejor llamar a una función externa.

Figura 1: Ejemplo de llamada a herramienta de un agente del clima

Tipos de herramientas

En un sistema de IA, una herramienta se define igual que una función en un programa no IA. La definición de la herramienta declara un contrato entre el modelo y la herramienta. Como mínimo, esto incluye un nombre claro, parámetros y una descripción en lenguaje natural que explica su propósito y cómo debe usarse. Las herramientas vienen en varios tipos diferentes; tres tipos principales descritos aquí son Herramientas de Función, Herramientas Integradas y Herramientas de Agente.

Herramientas de Función

Todos los modelos que soportan llamada a funciones¹⁰ permiten al desarrollador definir funciones externas que el modelo puede llamar según sea necesario. La definición de la herramienta debe proporcionar detalles básicos sobre cómo el modelo debe usar la herramienta; esto se proporciona al modelo como parte del contexto de la solicitud. En un framework de Python como Google ADK, la definición pasada al modelo se extrae del docstring de Python en el código de la herramienta como en el siguiente ejemplo.

Este ejemplo muestra una herramienta definida para Google ADK¹¹ que llama a una función externa para cambiar el brillo de una luz. El `set_light_values` se pasa un objeto `ToolContext` (parte del framework de Google ADK) para proporcionar más detalles sobre el contexto de la solicitud.

Fragmento 1: Definición para la herramienta `set_light_values`

```

python
def set_light_values(
    brightness: int,
    color_temp: str,
    context: ToolContext) -> dict[str, int | str]:
    """Esta herramienta establece el brillo y la temperatura de color de las luces de la habitación
    en la ubicación actual del usuario.
    Args:
        brightness: Nivel de luz de 0 a 100. Cero es apagado y 100 es brillo completo.
        color_temp: Temperatura de color de la luminaria, que puede ser `daylight`, `cool` o
        `warm`.
        context: Un objeto ToolContext utilizado para recuperar la ubicación del usuario.
    Returns:
        Un diccionario que contiene el brillo y la temperatura de color establecidos.
    """
    user_room_id = context.state['room_id']
    # Esta es una API imaginaria de control de iluminación
    room = light_system.get_room(user_room_id)
    response = room.set_lights(brightness, color_temp)
    return {"tool_response": response}

```

Herramientas integradas

Algunos modelos fundamentales ofrecen la capacidad de aprovechar herramientas integradas, donde la definición de la herramienta se da al modelo implícitamente, o entre bastidores del servicio del modelo. La API de Gemini de Google, por ejemplo, proporciona varias herramientas integradas: Grounding con Google Search¹², Code Execution¹³, URL Context¹⁴ y Computer Use¹⁵.

El siguiente ejemplo muestra cómo invocar la herramienta integrada `url\_context` de Gemini. La definición de la herramienta en sí es invisible para el desarrollador; se proporciona al modelo por separado.

Fragmento 2: Llamada a la herramienta url_context

```

python
from google import genai
from google.genai.types import (
    Tool,
    GenerateContentConfig,
    HttpOptions,
    UrlContext
)
client = genai.Client(http_options=HttpOptions(api_version="v1"))

```

```

model_id = "gemini-2.5-flash"
url_context_tool = Tool(
    url_context = UrlContext
)

url1 = "https://www.foodnetwork.com/recipes/ina-garten/perfect-roast-chicken-recipe-1940592"
url2 = "https://www.allrecipes.com/recipe/70679/simple-whole-roasted-chicken/"

response = client.models.generate_content(
    model=model_id,
    contents=("Compara los ingredientes y los tiempos de cocción de "
              f"las recetas en {url1} y {url2}"),
    config=GenerateContentConfig(
        tools=[url_context_tool],
        response_modalities=["TEXT"],
    )
)
for each in response.candidates[0].content.parts:
    print(each.text)
# Para verificación, puedes inspeccionar los metadatos para ver qué URLs el modelo recuperó
print(response.candidates[0].url_context_metadata)
...

```

Herramientas de Agente

Un agente también puede invocarse como una herramienta. Esto evita una transferencia completa de la conversación del usuario, permitiendo que el agente principal mantenga el control sobre la interacción y procese la entrada y salida del subagente según sea necesario. En ADK, esto se logra usando la clase `AgentTool`¹⁶ en el SDK. El protocolo A2A de Google¹⁷, discutido en el Día 5: De Prototipo a Producción, incluso te permite hacer que agentes remotos estén disponibles como herramientas.

Fragmento 3: Definición de AgentTool

```

```python
from google.adk.agents import LlmAgent
from google.adk.tools import AgentTool
tool_agent = LlmAgent(
 model="gemini-2.5-flash",
 name="capital_agent",
 description="Devuelve la ciudad capital para cualquier país o estado",
 instruction="""Si el usuario te da el nombre de un país o un estado (p. ej.,
 Tennessee o Nueva Gales del Sur), responde con el nombre de la ciudad capital de ese
 país o estado. De lo contrario, dile al usuario que no puedes ayudarlo."""
)

```

```
user_agent = LlmAgent(
 model="gemini-2.5-flash",
 name="user_advice_agent",
 description="Responde preguntas del usuario y da consejos",
 instruction="""Usa las herramientas que tienes disponibles para responder
a las preguntas del usuario""",
 tools=[AgentTool(agent=capital_agent)]
)
...
```

### Taxonomía de Herramientas de Agente

Una forma de categorizar las herramientas de agente es por su función principal, o los varios tipos de interacciones que facilitan. Aquí tienes una descripción general de los tipos comunes:

- **Recuperación de Información:** Permiten que los agentes obtengan datos de varias fuentes, como búsquedas web, bases de datos o documentos no estructurados.
- **Acción / Ejecución:** Permiten que los agentes realicen operaciones del mundo real: enviar correos electrónicos, publicar mensajes, iniciar la ejecución de código o controlar dispositivos físicos.
- **Integración de Sistema / API:** Permiten que los agentes se conecten con sistemas de software y API existentes, se integren en flujos de trabajo empresariales o interactúen con servicios de terceros.
- **Humano en el Bucle (Human-in-the-Loop):** Facilitan la colaboración con usuarios humanos: pedir aclaraciones, buscar aprobación para acciones críticas o transferir tareas para el juicio humano.

Tabla 1: Categorías de herramientas y consideraciones de diseño

Caso de Uso de Herramienta	Consejos Clave de Diseño
---	---
<b>Recuperación de Datos Estructurados</b>	Consultar bases de datos, hojas de cálculo u otras fuentes de datos estructurados (p. ej., MCP Toolbox, NL2SQL)   Define esquemas claros, optimiza para consultas eficientes, maneja los tipos de datos con elegancia.
<b>Recuperación de Datos No Estructurados</b>	Buscar documentos, páginas web o bases de conocimiento (p. ej., muestra RAG)   Implementa algoritmos de búsqueda robustos, considera las limitaciones de la ventana de contexto y proporciona instrucciones de recuperación claras.
<b>Conexión a Plantillas Integradas</b>	Generar contenido a partir de plantillas predefinidas   Asegúrate de que los parámetros de la plantilla estén bien definidos, proporciona una guía clara sobre la selección de plantillas.
<b>Conectores de Google</b>	Interactuar con aplicaciones de Google Workspace (p. ej., Gmail, Drive, Calendar)   Aprovecha las APIs de Google, asegúrate de una autenticación y autorización adecuadas, maneja los límites de tasa de la API.

| **\*\*Conectores de Terceros\*\*** | Integración con servicios y aplicaciones externos | Documenta las especificaciones de la API externa, gestiona las claves de API de forma segura, implementa el manejo de errores para llamadas externas. |

## ## Mejores Prácticas

A medida que el uso de herramientas se generaliza en las aplicaciones de IA y surgen nuevas categorías de herramientas, las mejores prácticas reconocidas para el uso de herramientas están evolucionando rápidamente. Sin embargo, varias pautas están emergiendo que parecen ampliamente aplicables.

### ### La documentación es importante

La documentación de la herramienta (nombre, descripción y atributos) se pasa toda al modelo como parte del contexto de la solicitud, por lo que todas estas son importantes para ayudar al modelo a usar la herramienta correctamente.

- **\*\*Usa un nombre claro:\*\*** El nombre de la herramienta debe ser claramente descriptivo, legible por humanos y específico para ayudar al modelo a decidir qué herramienta usar. Por ejemplo, ``create_critical_bug_in_jira_with_priority`` es más claro que ``update_jira``. Esto también es importante para la gobernanza; si las llamadas a herramientas se registran, tener nombres claros hará que los registros de auditoría sean más informativos.
- **\*\*Describe todos los parámetros de entrada y salida:\*\*** Todas las entradas a la herramienta deben describirse claramente, incluyendo tanto el tipo requerido como el uso que la herramienta hará del parámetro.
- **\*\*Simplifica las listas de parámetros:\*\*** Las listas de parámetros largas pueden confundir al modelo; mantén las listas de parámetros cortas y da a los parámetros nombres claros.
- **\*\*Aclara las descripciones de las herramientas:\*\*** Proporciona una descripción clara y detallada de los parámetros de entrada y salida, el propósito de la herramienta y cualquier otro detalle necesario para llamar a la herramienta de manera efectiva. Evita abreviaturas o jerga técnica; céntrate en explicaciones claras usando terminología simple.
- **\*\*Añade ejemplos dirigidos:\*\*** Los ejemplos pueden ayudar a abordar ambigüedades, mostrar cómo manejar solicitudes complicadas o aclarar distinciones en la terminología. También pueden ser una forma de refinar y dirigir el comportamiento del modelo sin recurrir a enfoques más costosos como el ajuste fino (fine-tuning). También puedes recuperar dinámicamente ejemplos relacionados con la tarea inmediata para minimizar la hinchazón del contexto.
- **\*\*Proporciona valores por defecto:\*\*** Proporciona valores por defecto para parámetros clave y asegúrate de documentar y describir los valores por defecto en la documentación de la herramienta. Los LLM a menudo pueden usar los valores por defecto correctamente, si están bien documentados.

Los siguientes son ejemplos de documentación de herramientas buena y mala.

#### Fragmento 4: Buena documentación de herramienta

```

```python
def get_product_information(product_id: str) -> dict:
    """
    Recupera información integral sobre un producto basado en el ID de producto único.
    Args:
        product_id: El identificador único para el producto.
    Returns:
        Un diccionario que contiene detalles del producto. Las claves esperadas incluyen:
        'product_name': El nombre del producto.
        'brand': El nombre de la marca del producto.
        'description': Un párrafo de texto que describe el producto.
        'category': La categoría del producto.
        'status': El estado actual del producto (p. ej., 'active', 'inactive', 'suspended').
    Ejemplo de valor de retorno:
    {
        'product_name': 'Astro Zoom Kid's Trainers',
        'brand': 'Cymbal Athletic Shoes',
        'description': '...',
        'category': 'Children's Shoes',
        'status': 'active'
    }
    """
```

```

#### Fragmento 5: Mala documentación de herramienta

```

```python
def fetchpd(pid):
    """
    Recupera datos del producto

    Args:
        pid: id
    Returns:
        diccionario de datos
    """
```

```

### Describir acciones, no implementaciones

Asumiendo que cada herramienta está bien documentada, las instrucciones del modelo deben describir acciones, no herramientas específicas. Esto es importante para eliminar cualquier posibilidad de conflicto entre las instrucciones sobre cómo usar la herramienta (lo que puede confundir al LLM). Donde las herramientas disponibles pueden cambiar dinámicamente, como con MCP, esto es aún más relevante.

- **\*\*Describe qué, no cómo:\*\*** Explica qué necesita hacer el modelo, no cómo hacerlo. Por ejemplo, di "crea un error para describir el problema", en lugar de "usa la herramienta `create_bug`".
- **\*\*No dupliques instrucciones:\*\*** No repitas o reiteres las instrucciones o documentación de la herramienta. Esto puede confundir al modelo y crea una dependencia adicional entre las instrucciones del sistema y la implementación de la herramienta.
- **\*\*No dictes flujos de trabajo:\*\*** Describe el objetivo y permite espacio para que el modelo use herramientas de forma autónoma, en lugar de dictar una secuencia específica de acciones.
- **\*\*EXPLICA las interacciones de las herramientas:\*\*** Si una herramienta tiene un efecto secundario que puede afectar a una herramienta diferente, documenta esto. Por ejemplo, una herramienta `fetch_web_page` puede almacenar la página web recuperada en un archivo; documenta esto para que el agente sepa cómo acceder a los datos.

#### Publicar tareas, no llamadas a la API

Las herramientas deben encapsular una tarea que el agente necesita realizar, no una API externa. Es fácil escribir herramientas que son simplemente envoltorios delgados sobre la superficie de API existente, pero esto es un error. En cambio, los desarrolladores de herramientas deben definir herramientas que capturen claramente acciones específicas que el agente podría tomar en nombre del usuario, y documentar la acción específica y los parámetros necesarios. Las API están destinadas a ser utilizadas por desarrolladores humanos con conocimiento completo de los datos disponibles y los parámetros de la API; las API empresariales complejas pueden tener decenas o incluso cientos de parámetros posibles que influyen en la salida de la API. Las herramientas para agentes, en cambio, se espera que sean usadas dinámicamente por un agente que necesita decidir en tiempo de ejecución qué parámetros usar y qué datos pasar. Si la herramienta representa una tarea específica que el agente debe cumplir, es mucho más probable que el agente pueda llamarla correctamente.

#### Haz las herramientas lo más granulares posible

Mantener las funciones concisas y limitadas a una única función es una mejor práctica de codificación estándar; sigue esta guía al definir herramientas también. Esto facilita la documentación de la herramienta y permite que el agente sea más consistente para determinar cuándo se necesita la herramienta.

- **\*\*Define responsabilidades claras:\*\*** Asegúrate de que cada herramienta tenga un propósito claro y bien documentado. ¿Qué hace? ¿Cuándo debe llamarse? ¿Tiene algún efecto secundario? ¿Qué datos devolverá?
- **\*\*No crees multi-herramientas:\*\*** En general, no crees herramientas que a su vez realicen muchos pasos o encapsulen un flujo de trabajo largo. Estas pueden ser complicadas de documentar y mantener, y pueden ser difíciles de que los LLM usen de manera consistente. Hay escenarios donde una herramienta de este tipo puede ser útil, por ejemplo, si un flujo de trabajo comúnmente realizado requiere muchas llamadas a herramientas en secuencia, definir

una única herramienta para encapsular muchas operaciones puede ser más eficiente. En estos casos, asegúrate de documentar muy claramente qué está haciendo la herramienta para que el LLM pueda usarla de manera efectiva.

#### #### Diseña para una salida concisa

Las herramientas mal diseñadas a veces pueden devolver grandes volúmenes de datos, lo que puede afectar adversamente el rendimiento y el costo.

- **\*\*No devuelvas respuestas grandes:\*\*** Tablas de datos grandes o diccionarios, archivos descargados, imágenes generadas, etc., pueden inundar rápidamente el contexto de salida de un LLM. Estas respuestas también se almacenan con frecuencia en el historial de conversación de un agente, por lo que las respuestas grandes pueden afectar las solicitudes posteriores también.
- **\*\*Usa sistemas externos:\*\*** Aprovecha los sistemas externos para el almacenamiento y acceso a datos. Por ejemplo, en lugar de devolver un resultado de consulta grande directamente al LLM, insértalo en una tabla de base de datos temporal y devuelve el nombre de la tabla, para que una herramienta posterior pueda recuperar los datos directamente. Algunos frameworks de IA también proporcionan almacenamiento externo persistente como parte del framework en sí, como el Artifact Service en Google ADK<sup>18</sup>.

#### #### Usa la validación de manera efectiva

La mayoría de los frameworks de llamada a herramientas incluyen validación de esquema opcional para entradas y salidas de herramientas. Usa esta capacidad de validación siempre que sea posible. Los esquemas de entrada y salida sirven a dos roles con la llamada a funciones de LLM. Sirven como documentación adicional de las capacidades y función de la herramienta, dando al LLM una imagen más clara de cuándo y cómo usar la herramienta; y proporcionan una verificación en tiempo de ejecución de la operación de la herramienta, permitiendo que la aplicación misma valide si la herramienta se está llamando correctamente.

#### #### Proporciona mensajes de error descriptivos

Los mensajes de error de las herramientas son una oportunidad pasada por alto para refinar y documentar las capacidades de las herramientas. A menudo, incluso las herramientas bien documentadas simplemente devolverán un código de error, o como mucho un mensaje de error corto y no descriptivo. En la mayoría de los sistemas de llamada a herramientas, la respuesta de la herramienta también se proporcionará al LLM que llama, por lo que proporciona otra vía para dar instrucciones. El mensaje de error de la herramienta también debería dar alguna instrucción al LLM sobre qué hacer para abordar el error específico. Por ejemplo, una herramienta que recupera datos de productos podría devolver una respuesta que diga "No se encontraron datos de productos para el ID de producto XXX. Pide al cliente que confirme el nombre del producto y busca el ID del producto por nombre para confirmar que tienes el ID correcto."

## ## Entendiendo el Protocolo de Contexto de Modelo

### ### El Problema de Integración "N x M" y la necesidad de Estandarización

Las herramientas proporcionan el enlace esencial entre un agente de IA o un LLM y el mundo exterior. Sin embargo, el ecosistema de herramientas, fuentes de datos y otras integraciones accesibles externamente es cada vez más fragmentado y complejo. Integrar un LLM con una herramienta externa generalmente requiere un conector personalizado, único para cada par de herramienta y aplicación. Esto lleva a una explosión en el esfuerzo de desarrollo, a menudo llamado el problema de integración "N x M", donde el número de conexiones personalizadas necesarias crece exponencialmente con cada nuevo modelo (N) o herramienta (M) añadido al ecosistema<sup>19</sup>.

Anthropic introdujo el Protocolo de Contexto de Modelo (MCP) en noviembre de 2024 como un estándar abierto para comenzar a abordar esta situación. El objetivo de MCP desde el principio ha sido reemplazar el panorama fragmentado de integraciones personalizadas con un protocolo unificado y de "conectar y usar" que pueda servir como una interfaz universal entre las aplicaciones de IA y el vasto mundo de herramientas y datos externos. Al estandarizar esta capa de comunicación, MCP tiene como objetivo desacoplar el agente de IA de los detalles de implementación específicos de las herramientas que utiliza, permitiendo un ecosistema más modular, escalable y eficiente.

### ### Componentes Arquitectónicos Centrales: Hosts, Clientes y Servidores

El Protocolo de Contexto de Modelo implementa un modelo cliente-servidor, inspirado en el Language Server Protocol (LSP) en el mundo del desarrollo de software<sup>9</sup>. Esta arquitectura separa la aplicación de IA de las integraciones de herramientas y permite un enfoque más modular y extensible para el desarrollo de herramientas. Los componentes centrales de MCP son el Host, el Cliente y el Servidor.

- **\*\*Host MCP:\*\*** La aplicación responsable de crear y gestionar clientes MCP individuales; puede ser una aplicación independiente o un subcomponente de un sistema más grande como un sistema multiagente. Las responsabilidades incluyen gestionar la experiencia del usuario, orquestar el uso de herramientas y hacer cumplir las políticas de seguridad y barreras de contenido.
- **\*\*Cliente MCP:\*\*** Un componente de software incrustado dentro del Host que mantiene la conexión con el Servidor. Las responsabilidades del cliente son emitir comandos, recibir respuestas y gestionar el ciclo de vida de la sesión de comunicación con su Servidor MCP.
- **\*\*Servidor MCP:\*\*** Un programa que proporciona un conjunto de capacidades que el desarrollador del servidor quiere poner a disposición de las aplicaciones de IA, funcionando a menudo como un adaptador o un proxy para una herramienta externa, fuente de datos o API. Las responsabilidades principales son anunciar las herramientas disponibles (descubrimiento de herramientas), recibir y ejecutar comandos, y formatear y devolver resultados. En contextos

empresariales, los servidores también son responsables de la seguridad, escalabilidad y gobernanza.

El siguiente diagrama muestra las relaciones entre cada uno de estos componentes y cómo se comunican.

Figura 2: Host, Cliente y Servidor MCP en una Aplicación de Agentes

Este modelo arquitectónico tiene como objetivo apoyar el desarrollo de un ecosistema de herramientas de IA competitivo e innovador. Los desarrolladores de agentes de IA deberían poder centrarse en su competencia principal: el razonamiento y la experiencia del usuario, mientras que los desarrolladores de terceros pueden crear servidores MCP especializados para cualquier herramienta o API concebible.

#### ### La Capa de Comunicación: JSON-RPC, Transportes y Tipos de Mensaje

Toda la comunicación entre clientes y servidores MCP se construye sobre una base técnica estandarizada para la coherencia e interoperabilidad.

**\*\*Protocolo Base:\*\*** MCP utiliza JSON-RPC 2.0 como su formato de mensaje base. Esto le da una estructura ligera, basada en texto y agnóstica del idioma para todas las comunicaciones.

**\*\*Tipos de Mensaje:\*\*** El protocolo define cuatro tipos de mensajes fundamentales que rigen el flujo de interacción:

- **\*\*Solicitudes (Requests):\*\*** Una llamada RPC enviada de una parte a otra que espera una respuesta.
- **\*\*Resultados (Results):\*\*** Un mensaje que contiene el resultado exitoso de una solicitud correspondiente.
- **\*\*Errores (Errors):\*\*** Un mensaje que indica que una solicitud falló, incluyendo código y descripción.
- **\*\*Notificaciones (Notifications):\*\*** Un mensaje unidireccional que no requiere respuesta y no puede ser respondido.

**\*\*Mecanismos de Transporte:\*\*** MCP también necesita un protocolo estándar para la comunicación entre el cliente y el servidor, llamado "protocolo de transporte", para asegurar que cada componente sea capaz de interpretar los mensajes del otro. MCP soporta dos protocolos de transporte: uno para comunicación local y otro para conexiones remotas<sup>20</sup>.

- **\*\*stdio (Entrada/Salida Estándar):\*\*** Se utiliza para comunicación rápida y directa en entornos locales donde el servidor MCP se ejecuta como un subproceso de la aplicación Host; se usa cuando las herramientas necesitan acceder a recursos locales como el sistema de archivos del usuario.
- **\*\*HTTP Transmisible (Streamable HTTP):\*\*** Protocolo cliente-servidor remoto recomendado<sup>21</sup>. Soporta respuestas de streaming SSE, pero también permite servidores sin estado y puede implementarse en un servidor HTTP plano sin requerir SSE.

Figura 3: Protocolos de Transporte MCP

### ### Primitivas Clave: Herramientas y otras

Sobre el marco de comunicación básico, MCP define varios conceptos clave o tipos de entidad para mejorar las capacidades de las aplicaciones basadas en LLM para interactuar con sistemas externos. Las tres primeras son capacidades ofrecidas por el Servidor al Cliente; las tres restantes son ofrecidas por el Cliente al servidor. En el lado del servidor, estas capacidades son: Herramientas, Recursos e Instrucciones; y en el lado del cliente, las capacidades son Muestreo, Elicitación y Raíces.

De estas capacidades definidas por la especificación MCP, solo las Herramientas tienen un amplio soporte. Como muestra la tabla a continuación, aunque las Herramientas son soportadas por casi todas las aplicaciones cliente rastreadas, los Recursos y las Instrucciones solo son soportados por aproximadamente un tercio, y el soporte para capacidades del lado del cliente es significativamente menor. Por lo tanto, queda por ver si estas capacidades jugarán un papel importante en los despliegues futuros de MCP.

Tabla 2: Porcentaje de clientes MCP públicamente disponibles que soportan capacidades de servidor/cliente MCP. Fuente: <https://modelcontextprotocol.io/clients>, recuperado el 15 de septiembre de 2025

| Capacidad   Estado de Soporte del Cliente   % Soportado |  |  |  |
|---------------------------------------------------------|--|--|--|
| :---   :---   :---                                      |  |  |  |
| Soportado   No soportado   Desconocido/Otro             |  |  |  |
| <b>Herramientas</b>   78   1   0   99%                  |  |  |  |
| <b>Recursos</b>   27   51   1   34%                     |  |  |  |
| <b>Instrucciones</b>   25   54   0   32%                |  |  |  |
| <b>Muestreo</b>   8   70   1   10%                      |  |  |  |
| <b>Elicitación</b>   3   74   2   4%                    |  |  |  |
| <b>Raíces</b>   4   75   0   5%                         |  |  |  |

En esta sección nos centraremos en las Herramientas, ya que tienen con diferencia la adopción más amplia y son el motor principal del valor de MCP, y solo describiremos brevemente las capacidades restantes.

### #### Herramientas

La entidad Herramienta22 en MCP es una forma estandarizada para que un servidor describa una función que pone a disposición de los clientes. Algunos ejemplos podrían ser `read\_file`, `get\_weather`, `execute\_sql` o `create\_ticket`. Los Servidores MCP publican una lista de sus herramientas disponibles, incluyendo descripciones y esquemas de parámetros, para que los agentes las descubran.

## ##### Definición de Herramienta

Las definiciones de herramientas deben conformarse a un esquema JSON23 con los siguientes campos:

- **\*\*name:\*\*** Identificador único para la herramienta
- **\*\*title:\*\*** [OPCIONAL] nombre legible por humanos para fines de visualización
- **\*\*description:\*\*** Descripción legible por humanos (y por LLM) de la funcionalidad
- **\*\*inputSchema:\*\*** Esquema JSON que define los parámetros esperados de la herramienta
- **\*\*outputSchema:\*\*** [OPCIONAL]: Esquema JSON que define la estructura de salida
- **\*\*annotations:\*\*** [OPCIONAL]: Propiedades que describen el comportamiento de la herramienta

La documentación de las herramientas en MCP debería seguir las mismas mejores prácticas generales que describimos anteriormente. Por ejemplo, las propiedades como ``title`` y ``description`` pueden ser opcionales en el esquema, pero siempre deberían incluirse. Proporcionan un canal importante para dar instrucciones más detalladas a los LLM clientes sobre cómo usar la herramienta de manera efectiva.

Los campos ``inputSchema`` y ``outputSchema`` también son críticos para asegurar el uso correcto de la herramienta. Deberían ser descriptivos y cuidadosamente redactados, y cada propiedad definida en ambos esquemas debería tener un nombre descriptivo y una descripción clara. Ambos campos de esquema deberían tratarse como obligatorios.

El campo ``annotations`` se declara como opcional y debería permanecer así. Las propiedades definidas en la especificación son:

- **\*\*destructiveHint:\*\*** Puede realizar actualizaciones destructivas (por defecto: true).
- **\*\*idempotentHint:\*\*** Llamar repetidamente con los mismos argumentos no tendrá efecto adicional (por defecto: false).
- **\*\*openWorldHint:\*\*** Puede interactuar con un "mundo abierto" de entidades externas (por defecto: true).
- **\*\*readOnlyHint:\*\*** No modifica su entorno (por defecto: false)
- **\*\*title:\*\*** Título legible por humanos para la herramienta (nota que esto no necesita coincidir con el título proporcionado en la definición de la herramienta).

Todas las propiedades declaradas en este campo son solo sugerencias, y no se garantiza que describan con precisión las operaciones de la herramienta. Los clientes MCP no deberían confiar en estas propiedades de servidores no confiables, e incluso cuando el servidor es de confianza, la especificación no requiere que las propiedades de la herramienta estén garantizadas como verdaderas. Ejecuta precaución al hacer uso de estas anotaciones.

El siguiente ejemplo muestra una Definición de Herramienta MCP con cada uno de estos campos incluido.

JSON

```

{
 "name": "get_stock_price",
 "title": "Herramienta de Recuperación de Precios de Acciones",
 "description": "Obtiene el precio de una acción para un símbolo bursátil específico. Si se proporciona 'fecha', recuperará el último precio o precio de cierre para esa fecha. De lo contrario, recuperará el precio más reciente.",
 "inputSchema": {
 "type": "object",
 "properties": {
 "symbol": {
 "type": "string",
 "description": "Símbolo bursátil",
 }
 },
 "date": {
 "type": "string",
 "description": "Fecha a recuperar (en formato AAAA-MM-DD)"
 }
 },
 "required": ["symbol"]
}

```

Continúa en la siguiente página...

Fragmento 6: Ejemplo de definición de herramienta para una herramienta de recuperación de precios de acciones

```

```json
{
  "name": "get_stock_price",
  "title": "Herramienta de Recuperación de Precios de Acciones",
  "description": "Obtiene el precio de una acción para un símbolo bursátil específico. Si se proporciona 'fecha', recuperará el último precio o precio de cierre para esa fecha. De lo contrario, recuperará el precio más reciente.",
  "inputSchema": {
    "type": "object",
    "properties": {
      "symbol": {
        "type": "string",
        "description": "Símbolo bursátil"
      }
    },
    "date": {
      "type": "string",
      "description": "Fecha a recuperar (en formato AAAA-MM-DD)"
    }
  },
  "required": ["symbol"]
}

```

```

"outputSchema": {
  "type": "object",
  "properties": {
    "price": {
      "type": "number",
      "description": "Precio de la acción"
    },
    "date": {
      "type": "string",
      "description": "Fecha del precio de la acción"
    }
  },
  "required": ["price", "date"]
},
"annotations": {
  "readOnlyHint": "true"
}
}
...

```

Resultados de Herramienta

Las herramientas MCP pueden devolver sus resultados de varias maneras. Los resultados pueden ser estructurados o no estructurados, y pueden contener múltiples tipos de contenido diferentes. Los resultados pueden enlazar a otros recursos en el servidor, y los resultados también pueden devolverse como una única respuesta o un flujo de respuestas.

****Contenido No Estructurado****

El contenido no estructurado puede tomar varios tipos. El tipo Texto representa datos de cadena no estructurados; los tipos de contenido Audio e Imagen contienen datos de imagen o audio codificados en base64 etiquetados con el tipo MIME apropiado.

MCP también permite que las Herramientas devuelvan Recursos especificados, lo que da a los desarrolladores más opciones para gestionar su flujo de trabajo de la aplicación. Los Recursos pueden devolverse como un enlace a una entidad Recurso almacenada en otro URI, incluyendo el título, descripción, tamaño y tipo MIME; o completamente incrustados en el Resultado de la Herramienta. En cualquier caso, los desarrolladores clientes deberían ser muy cautelosos al recuperar o usar recursos devueltos de un servidor MCP de esta manera, y solo deberían usar Recursos de fuentes confiables.

****Contenido Estructurado****

El contenido estructurado siempre se devuelve como un objeto JSON. Los implementadores de herramientas siempre deberían usar la capacidad `outputSchema` para proporcionar un esquema JSON que los clientes puedan usar para validar los resultados de la herramienta, y

los desarrolladores clientes deberían validar los resultados de la herramienta contra el esquema proporcionado. Al igual que con la llamada a funciones estándar, un esquema de salida definido sirve a un doble propósito: permite al cliente interpretar y analizar la salida de manera efectiva, y comunica al LLM que llama cómo y por qué usar esta herramienta en particular.

Manejo de Errores

MCP también define dos mecanismos estándar de reporte de errores. Un Servidor puede devolver errores JSON-RPC estándar para problemas de protocolo como herramientas desconocidas, argumentos inválidos o errores del servidor. También puede devolver mensajes de error en los resultados de la herramienta estableciendo el parámetro `"isError": true` en el objeto de resultado. Estos errores se usan para errores generados en la operación de la herramienta en sí, como fallos de API de backend, datos inválidos o errores de lógica de negocio. Los mensajes de error son un canal importante y a menudo pasado por alto para proporcionar contexto adicional al LLM que llama. Los desarrolladores de herramientas MCP deberían considerar cómo usar mejor este canal para ayudar a sus clientes a recuperarse de errores. Los siguientes ejemplos muestran cómo un desarrollador podría usar cada uno de estos tipos de error para proporcionar orientación adicional al LLM cliente.

Fragmento 7: Ejemplo de error de protocolo. Fuente:

<https://modelcontextprotocol.io/specification/2025-06-18/server/tools#error-handling>, recuperado el 16-09-2025.

```
```json
{
 "jsonrpc": "2.0",
 "id": 3,
 "error": {
 "code": -32602,
 "message": "Herramienta desconocida: invalid_tool_name. Puede estar mal escrita, o la herramienta puede no existir en este servidor. Verifica el nombre de la herramienta y si es necesario solicita una lista actualizada de herramientas."
 }
}
```
```

Fragmento 8: Ejemplo de error de ejecución de herramienta. Fuente:

<https://modelcontextprotocol.io/specification/2025-06-18/server/tools#error-handling>, recuperado el 16-09-2025.

```
```json
{
 "jsonrpc": "2.0",
 "id": 4,
 "result": {
```

```

"content": [
 {
 "type": "text",
 "text": "Error al obtener los datos del clima: límite de tasa de la API excedido. Espera 15 segundos antes de volver a llamar a esta herramienta."
 },
],
 "isError": true
}
...

```

### ### Otras Capacidades

Además de las Herramientas, la especificación MCP define otras cinco capacidades que los servidores y clientes pueden proporcionar. Como señalamos anteriormente, solo un pequeño número de implementaciones MCP soportan estas capacidades, por lo que queda por ver si jugarán un papel importante en los despliegues basados en MCP.

#### #### Recursos

Los Recursos<sup>24</sup> son una capacidad del lado del servidor destinada a proporcionar datos contextuales que pueden ser accedidos y utilizados por la aplicación Host. Los recursos proporcionados por un servidor MCP podrían incluir el contenido de un archivo, un registro de una base de datos, un esquema de base de datos, una imagen u otra pieza de información de datos estáticos que los desarrolladores del servidor pretenden que sea utilizada por un cliente.

Ejemplos comúnmente citados de posibles Recursos incluyen archivos de registro, datos de configuración, estadísticas de mercado o blobs estructurados como PDFs o imágenes. Introducir contenido externo arbitrario en el contexto del LLM conlleva riesgos de seguridad significativos (ver más abajo), sin embargo, por lo que cualquier recurso consumido por un cliente LLM debería ser validado y recuperado de una URL confiable.

#### #### Instrucciones

Las Instrucciones<sup>25</sup> en MCP son otra capacidad del lado del servidor, permitiendo que el servidor proporcione ejemplos de instrucciones reutilizables o plantillas relacionadas con sus Herramientas y Recursos. Las Instrucciones están destinadas a ser recuperadas y utilizadas por el cliente para interactuar directamente con un LLM. Al proporcionar una Instrucción, un servidor MCP puede dar a sus clientes una descripción de más alto nivel de cómo usar las herramientas que proporciona.

Aunque tienen el potencial de añadir valor a un sistema de IA, en un entorno empresarial distribuido el uso de Instrucciones introduce algunas preocupaciones de seguridad evidentes.

Permitir que un servicio de terceros inyecte instrucciones arbitrarias en la ruta de ejecución de la aplicación es arriesgado, incluso cuando se filtra por clasificadores, auto-evaluadores u otros métodos de detección basados en LLM. En este momento, nuestra recomendación es que las Instrucciones deberían usarse raramente, si es que se usan, hasta que se desarrolle un modelo de seguridad más sólido.

#### #### Muestreo

El Muestreo<sup>26</sup> es una capacidad del lado del cliente que permite que un servidor MCP solicite una finalización de LLM al cliente. Si una de las capacidades del servidor necesita entrada de un LLM, en lugar de implementar la llamada LLM y usar los resultados internamente, el servidor emitiría una solicitud de Muestreo de vuelta al cliente para que el cliente la ejecute. Esto invierte el flujo de control típico, permitiendo que una herramienta aproveche el modelo de IA central del Host para realizar una subtarea, como pedirle al LLM que resuma un documento grande que el servidor acaba de recuperar. La especificación MCP recomienda que los clientes inserten una etapa de humano en el bucle en el Muestreo, de modo que siempre haya la opción de que un usuario deniegue una solicitud de Muestreo del servidor.

El Muestreo presenta tanto oportunidades como desafíos para los desarrolladores. Al externalizar las llamadas LLM al cliente, el Muestreo da a los desarrolladores clientes control sobre los proveedores de LLM utilizados en sus aplicaciones, y permite que los costos sean asumidos por el desarrollador de la aplicación en lugar del proveedor del servicio. El Muestreo también da al desarrollador cliente control sobre cualquier barrera de contenido y filtros de seguridad necesarios alrededor de la llamada LLM, y proporciona una manera limpia de insertar un paso de aprobación humana para solicitudes LLM que ocurren en la ruta de ejecución de la aplicación. Por otro lado, como la capacidad de Instrucciones, el Muestreo también abre una vía para una potencial inyección de instrucciones en la aplicación cliente. Los clientes deberían tener cuidado de filtrar y validar cualquier instrucción que acompañe a una solicitud de muestreo, y deberían asegurarse de que la fase de control de humano en el bucle se implemente con controles efectivos para que los usuarios interactúen con la solicitud de muestreo.

#### #### Elicitación

La Elicitación<sup>27</sup> es otra capacidad del lado del cliente, similar al Muestreo, que permite que un servidor MCP solicite información adicional del usuario al cliente. En lugar de solicitar una llamada LLM, una herramienta MCP que usa Elicitación puede consultar dinámicamente a la aplicación host para datos adicionales para completar la solicitud de la herramienta. La Elicitación proporciona un mecanismo formal para que un servidor pause una operación e interactúe con el usuario humano a través de la UI del cliente, permitiendo que el cliente mantenga el control de la interacción y el intercambio de datos, mientras que le da al servidor una forma de obtener entrada del usuario.

Los problemas de seguridad y privacidad son preocupaciones importantes en torno a esta capacidad. La especificación de MCP señala que "Los servidores NO DEBEN usar la elicitación para solicitar información sensible", y que los usuarios deberían ser claramente informados sobre el uso de la información y poder aprobar, declinar o cancelar la solicitud. Estas pautas son críticas para implementar la Elicitación de una manera que respete y preserve la privacidad y seguridad del usuario. La prohibición de solicitar información sensible es imposible de hacer cumplir de manera sistemática, por lo que los desarrolladores clientes necesitan estar vigilantes sobre el posible mal uso de esta capacidad. Si un cliente no proporciona barreras robustas alrededor de las solicitudes de elicitación y una interfaz clara para aprobar o denegar solicitudes, un desarrollador de servidor malicioso podría extraer fácilmente información sensible del usuario.

#### #### Raíces

Raíces, la tercera capacidad del lado del cliente, "definen los límites de dónde los servidores pueden operar dentro del sistema de archivos"<sup>28</sup>. Una definición de Raíz incluye un URI que identifica la raíz; en el momento de escribir, la especificación MCP restringe los URI de Raíz solo a URIs `file:`, pero esto puede cambiar en revisiones futuras. Se espera que un servidor que recibe una especificación de Raíz de un cliente confine sus operaciones solo a ese alcance. En la práctica, aún no está claro si o cómo se usarían las Raíces en un sistema MCP de producción. Por una parte, no hay barreras en la especificación en torno al comportamiento de los servidores con respecto a las Raíces, ya sea que la raíz sea un archivo local u otro tipo de URI. La declaración más clara sobre esto en la especificación es que "los servidores DEBERÍAN.. respetar los límites de la raíz durante las operaciones"<sup>29</sup>. Cualquier desarrollador cliente sería sabio en no confiar demasiado en el comportamiento del servidor con respecto a las Raíces.

#### ## Protocolo de Contexto de Modelo: A Favor y en Contra

MCP añade varias capacidades nuevas significativas a la caja de herramientas del desarrollador de IA. También tiene algunas limitaciones y desventajas importantes, particularmente a medida que su uso se expande desde el escenario de aumento de desarrollador desplegado localmente hasta aplicaciones de integración empresarial desplegadas remotamente. En esta sección primero veremos las ventajas y nuevas capacidades de MCP; luego consideramos las trampas, deficiencias, desafíos y riesgos que MCP introduce.

#### #### Capacidades y Ventajas Estratégicas

##### ##### Aceleración del Desarrollo y Fomento de un Ecosistema Reutilizable

El beneficio más inmediato de MCP está en simplificar el proceso de integración. MCP proporciona un protocolo común para la integración de herramientas con aplicaciones basadas

en LLM. Esto debería ayudar a reducir el costo de desarrollo, y por lo tanto el tiempo de comercialización, para nuevas características y soluciones impulsadas por IA.

MCP también puede ayudar a fomentar un ecosistema de "conectar y usar" donde las herramientas se conviertan en activos reutilizables y compartibles. Ya han surgido varios registros y mercados públicos de servidores MCP, que permiten a los desarrolladores descubrir, compartir y contribuir conectores preconstruidos. Para evitar la posible fragmentación del ecosistema MCP, el proyecto MCP lanzó recientemente el Registro MCP30, que proporciona tanto una fuente centralizada de verdad para servidores MCP públicos, como una especificación OpenAPI para estandarizar las declaraciones de servidores MCP. Si el registro MCP coge impulso, esto puede crear efectos de red que podrían acelerar el crecimiento del ecosistema de herramientas de IA.

#### #### Mejora Dinámica de las Capacidades y Autonomía de los Agentes

MCP mejora la llamada a funciones de agentes de varias maneras importantes.

- **\*\*Descubrimiento Dinámico de Herramientas:\*\*** Las aplicaciones habilitadas para MCP pueden descubrir las herramientas disponibles en tiempo de ejecución en lugar de tener esas herramientas codificadas, permitiendo una mayor adaptabilidad y autonomía.
- **\*\*Estandarización y Estructuración de Descripciones de Herramientas:\*\*** MCP también se expande sobre la llamada a funciones básica de LLM proporcionando un marco estándar para descripciones de herramientas y definiciones de interfaz.
- **\*\*Expansión de las Capacidades del LLM:\*\*** Finalmente, al permitir el crecimiento de un ecosistema de proveedores de herramientas, MCP expande dramáticamente las capacidades e información disponibles para los LLMs.

#### #### Flexibilidad Arquitectónica y Futuro-Prueba

Al estandarizar la interfaz agente-herramienta, MCP desacopla la arquitectura del agente de la implementación de sus capacidades. Esto promueve un diseño de sistema modular y componible, alineándose con paradigmas arquitectónicos modernos como la "malla de IA de agentes" (agentic AI mesh). En una arquitectura de este tipo, la lógica, la memoria y las herramientas se tratan como componentes independientes e intercambiables, haciendo que dichos sistemas sean más fáciles de depurar, actualizar, escalar y mantener a largo plazo. Dicha arquitectura modular también permite a una organización cambiar los proveedores de LLM subyacentes o reemplazar un servicio backend sin necesidad de re-arquitecturar toda la capa de integración, siempre que los nuevos componentes se expongan a través de un servidor MCP compatible.

#### #### Fundamentos para la Gobernanza y el Control

Si bien las características de seguridad nativas de MCP son actualmente limitadas (como se detalla en la siguiente sección), su arquitectura al menos proporciona los ganchos necesarios para implementar una gobernanza más robusta. Por ejemplo, las políticas de seguridad y los

controles de acceso pueden integrarse dentro del servidor MCP, creando un único punto de aplicación que asegura que cualquier agente que se conecte se adhiera a reglas predefinidas. Esto permite a una organización controlar qué datos y acciones se exponen a sus agentes de IA.

Además, la especificación del protocolo en sí establece una base filosófica para una IA responsable al recomendar explícitamente el consentimiento y el control del usuario. La especificación exige que los hosts obtengan la aprobación explícita del usuario antes de invocar cualquier herramienta o compartir datos privados. Este principio de diseño promueve la implementación de flujos de trabajo "humano en el bucle", donde el agente puede proponer una acción pero debe esperar la autorización humana antes de la ejecución, proporcionando una capa de seguridad crítica para los sistemas autónomos.

### ### Riesgos Críticos y Desafíos

Un enfoque clave para los desarrolladores empresariales que adoptan MCP es la necesidad de añadir soporte para requisitos de seguridad a nivel empresarial (autenticación, autorización, aislamiento de usuario, etc.). La seguridad es un tema tan crítico para MCP que dedicamos una sección separada de este documento blanco a ello (ver Sección 5). En el resto de esta sección, veremos otras consideraciones para desplegar MCP en aplicaciones empresariales.

### #### Cuellos de botella de Rendimiento y Escalabilidad

Más allá de la seguridad, el diseño actual de MCP presenta desafíos fundamentales para el rendimiento y la escalabilidad, principalmente relacionados con cómo gestiona el contexto y el estado.

- **\*\*Hinchazón de la Ventana de Contexto:\*\*** Para que un LLM sepa qué herramientas están disponibles, las definiciones y esquemas de parámetros de cada herramienta de cada servidor MCP conectado deben incluirse en la ventana de contexto del modelo. Estos metadatos pueden consumir una porción significativa del token disponible, resultando en un aumento del costo y la latencia, y causando la pérdida de otra información de contexto crítica.
- **\*\*Degradación de la Calidad del Razonamiento:\*\*** Una ventana de contexto sobrecargada también puede degradar la calidad del razonamiento de la IA. Con muchas definiciones de herramientas en una instrucción, el modelo puede tener dificultad para identificar la herramienta más relevante para una tarea dada o puede perder de vista la intención original del usuario. Esto puede llevar a un comportamiento errático, como ignorar una herramienta útil o invocar una irrelevante, o ignorar otra información importante contenida en el contexto de la solicitud.
- **\*\*Desafíos del Protocolo con Estado:\*\*** El uso de conexiones persistentes con estado para servidores remotos puede llevar a arquitecturas más complejas que son más difíciles de desarrollar y mantener. Integrar estas conexiones con estado con las APIs REST predominantemente sin estado a menudo requiere que los desarrolladores construyan y gestionen capas complejas de gestión de estado, lo que puede dificultar la escalabilidad horizontal y el equilibrio de carga.

El problema de la hinchazón de la ventana de contexto representa un desafío arquitectónico emergente: el paradigma actual de precargar todas las definiciones de herramientas en la instrucción es simple pero no escala. Esta realidad puede forzar un cambio en cómo los agentes descubren y utilizan las herramientas. Una arquitectura futura potencial podría implicar un enfoque similar a RAG para el descubrimiento de herramientas en sí<sup>31</sup>. Un agente, ante una tarea, realizaría primero un paso de "recuperación de herramientas" contra una biblioteca masiva e indexada de todas las herramientas posibles para encontrar las más relevantes. Basándose en esa respuesta, cargaría las definiciones de ese pequeño subconjunto de herramientas en su ventana de contexto para su ejecución.

Esto transformaría el descubrimiento de herramientas de un proceso de carga estática y por fuerza bruta a un problema de búsqueda dinámico, inteligente y escalable, creando una nueva y necesaria capa en la pila de IA de agentes. Sin embargo, la recuperación dinámica de herramientas abre otro posible vector de ataque; si un atacante obtiene acceso al índice de recuperación, podría inyectar un esquema de herramienta malicioso en el índice y engañar al LLM para que llame a una herramienta no autorizada.

#### #### Brechas de Preparación Empresarial

Si bien MCP se está adoptando rápidamente, varias características críticas a nivel empresarial aún están evolucionando o no se incluyen en el protocolo central, creando brechas que las organizaciones deben abordar por sí mismas.

- **\*\*Autenticación y Autorización:\*\*** La especificación MCP inicial no incluía originalmente un estándar robusto y listo para la empresa para autenticación y autorización. Si bien la especificación está evolucionando activamente, se ha señalado que la implementación de OAuth actual entra en conflicto con algunas prácticas de seguridad empresariales modernas<sup>32</sup>.
- **\*\*Ambigüedad en la Gestión de Identidades:\*\*** El protocolo aún no tiene una manera clara y estandarizada de gestionar y propagar la identidad. Cuando se realiza una solicitud, puede ser ambiguo si la acción está siendo iniciada por el usuario final, el agente de IA en sí, o una cuenta de sistema genérica. Esta ambigüedad complica la auditoría, la rendición de cuentas y la aplicación de controles de acceso granular.
- **\*\*Falta de Observabilidad Nativa:\*\*** El protocolo base no define estándares para primitivas de observabilidad como registro, trazas y métricas, capacidades esenciales para la depuración, monitoreo de salud y detección de amenazas. Para abordar esto, los proveedores de software empresarial están construyendo características sobre MCP con ofertas como la plataforma de gestión de APIs de Apigee, que añade una capa de observabilidad y gobernanza al tráfico MCP.

MCP fue diseñado para la innovación abierta y descentralizada, lo que impulsó su rápido crecimiento, y en el escenario de despliegue local, este enfoque tiene éxito. Sin embargo, los riesgos más significativos que presenta: vulnerabilidades de la cadena de suministro, seguridad inconsistente, fugas de datos y falta de observabilidad, son todas consecuencias de este

modelo descentralizado. Como resultado, los principales actores empresariales no están adoptando el protocolo "puro" sino que lo están envolviendo en capas de gobernanza centralizada. Estas plataformas gestionadas imponen la seguridad, identidad y control que extienden el protocolo base.

## ## Seguridad en MCP

### ### Nuevo panorama de amenazas

Junto con las nuevas capacidades que MCP ofrece al conectar agentes con herramientas y recursos, llega un nuevo conjunto de desafíos de seguridad que van más allá de las vulnerabilidades de aplicaciones tradicionales<sup>33</sup>. Los riesgos introducidos por MCP resultan de dos consideraciones paralelas: MCP como una nueva superficie de API, y MCP como un protocolo estándar.

Como una nueva superficie de API, el protocolo MCP base no incluye inherentemente muchas de las características y controles de seguridad implementados en endpoints de API tradicionales y otros sistemas. Exponer APIs existentes o sistemas de backend a través de MCP puede llevar a nuevas vulnerabilidades...